

第三章 Linux底层（面试突击★★★★★）

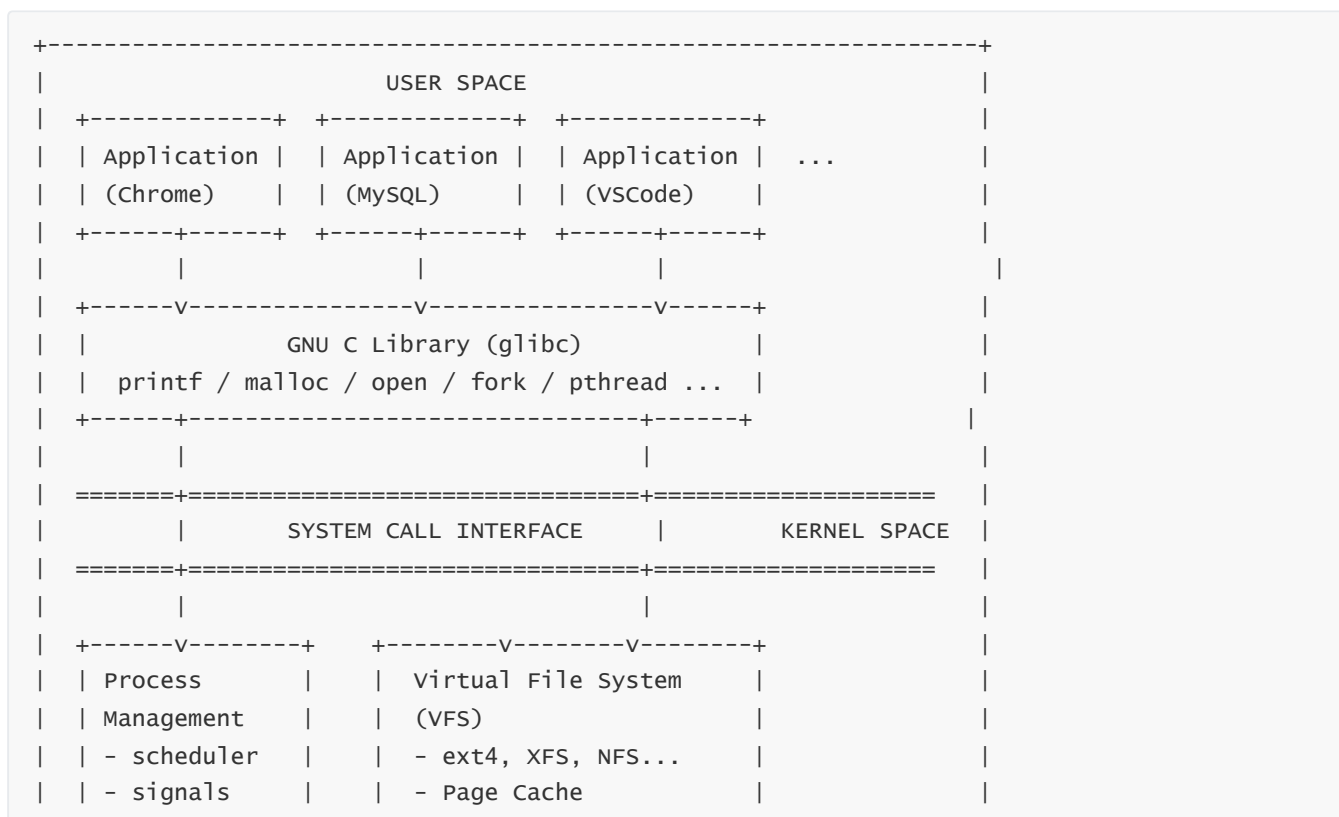
阅读指南: 本章是面试中最高频的考察模块之一，特别是进程管理、内存管理、文件系统这三个板块。建议结合strace和gdb实操加深理解。

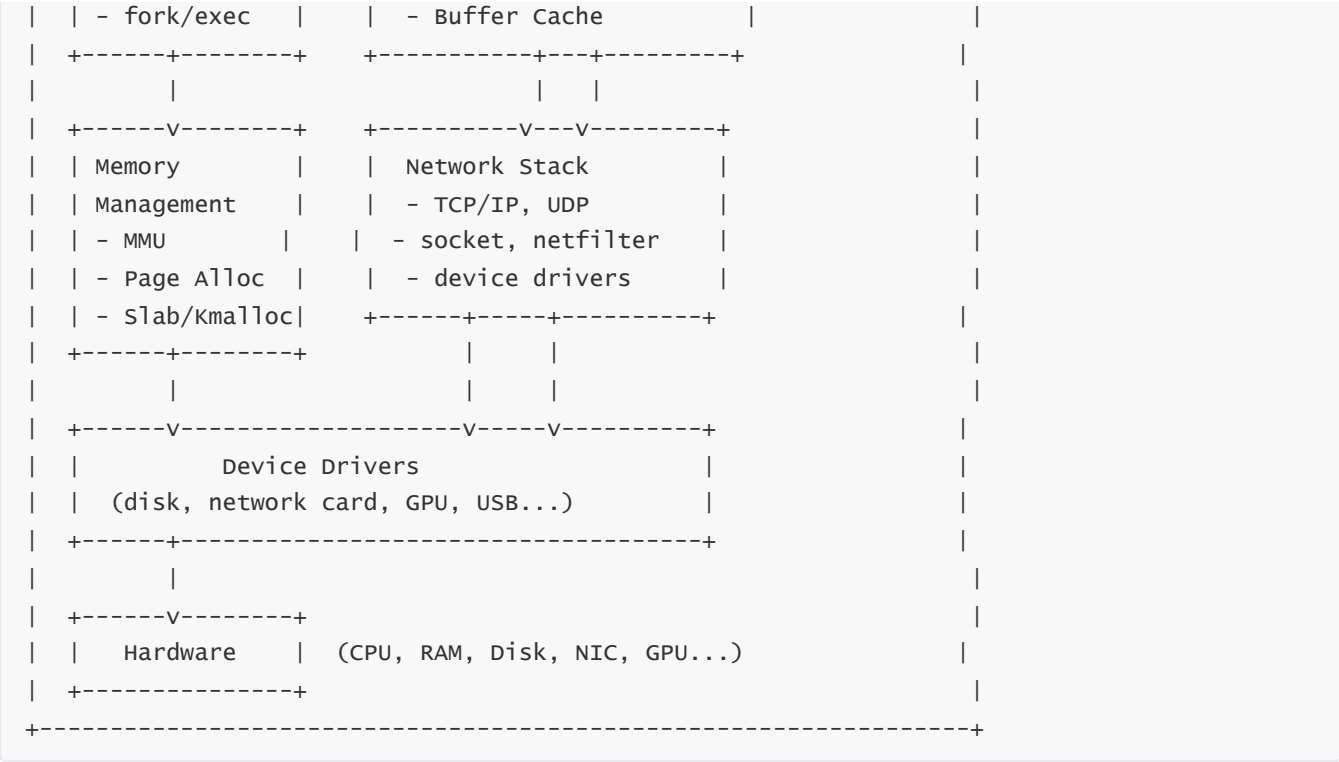
目录

1. [Linux架构](#)
2. [用户态/内核态](#)
3. [系统调用](#)
4. [进程管理](#)
5. [线程](#)
6. [内存管理](#)
7. [malloc/free](#)
8. [Copy On Write \(COW\)](#)
9. [文件系统](#)
10. [IO流程](#)
11. [面试重点汇总](#)

1. Linux架构

1.1 整体架构图





1.2 内核空间 vs 用户空间

维度	用户空间 (User Space)	内核空间 (Kernel Space)
运行权限	Ring 3 (最低特权级)	Ring 0 (最高特权级)
地址空间	每个进程独立的虚拟地址空间 (0x00000000 ~ 0xBFFFFFFF 典型)	所有进程共享的高地址区域 (0xC0000000 ~ 0xFFFFFFFF 典型 32位)
直接访问硬件	不能	可以
可执行指令	不能执行特权指令 (如 lgdt, lidt, hlt, cli, sti)	可以执行所有指令
崩溃影响	仅影响当前进程 (Segfault)	整个系统崩溃 (Kernel Panic / Oops)
内存保护	MMU强制隔离, 进程A不能访问进程B的内存	可访问所有物理内存
典型组件	应用程序, glibc, systemd, sshd	调度器, 内存管理, 驱动, 网络协议栈

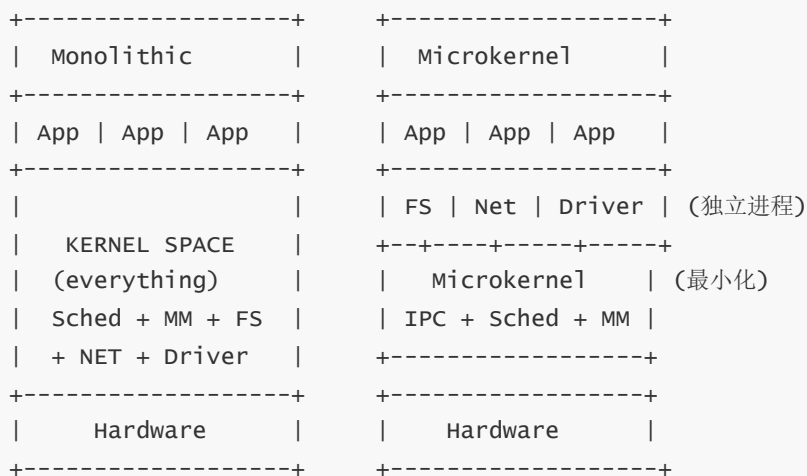
1.3 宏内核 (Monolithic Kernel) 设计

Linux 采用的是**宏内核**架构，与之相对的是**微内核** (Microkernel, 如 Minix, QNX, L4)。

宏内核特点:

- 所有内核服务 (进程调度、内存管理、文件系统、网络栈、设备驱动) 都在同一个内核地址空间运行
- 内核模块之间通过**函数调用**直接通信，高效但耦合度高
- 使用内核模块 (`.ko` 文件) 实现可扩展性，运行时动态加载/卸载

宏内核 vs 微内核对比：



优点：性能高（函数调用 vs IPC） 优点：稳定性好，安全隔离

缺点：一个驱动崩溃可能整个挂 缺点：IPC开销大，性能较低

Linux 是**混合型**的：宏内核 + 可加载内核模块 (LKM)，兼具性能和灵活性。内核编译时可以 `[*]` (built-in) 或 `[M]` (module) 选择功能。

2. 用户态/内核态

2.1 什么是用户态和内核态

CPU为了保护系统安全，设计了多个特权级别 (Ring 0 ~ Ring 3)。x86架构使用 Ring 0 和 Ring 3:

- **内核态 (Ring 0)**: CPU可以执行所有指令，访问所有内存地址，直接操作硬件。操作系统内核运行在此级别。
- **用户态 (Ring 3)**: CPU不能执行特权指令，只能访问受限的地址空间。普通应用程序运行在此级别。

CS寄存器的最低两位 (CPL, Current Privilege Level) 指示当前CPU处于哪个特权级:

- CPL = 0: 内核态
- CPL = 3: 用户态

2.2 用户态/内核态切换的三种方式

方式一: 系统调用 (System Call)

用户程序主动请求内核服务 (如 read, write, fork) 时触发。

```
用户程序调用 printf("hello")
      |
      v
glibc中的printf最终调用 write(1, buf, len)
      |
      v
glibc封装函数:
- 把系统调用号 (__NR_write = 1) 放入 eax/rax
- 把参数放入 rdi, rsi, rdx, r10, r8, r9
```

```

- 执行 syscall 指令 (x86-64) 或 int 0x80 (x86-32)
  |
  v
[ CPU切换到Ring 0, 内核态 ]
  |
  v
内核的 sys_call_table 根据 eax 找到 sys_write()
  |
  v
sys_write() 执行实际的内核逻辑
  |
  v
返回结果到 rax, 执行 sysret / iret 回到用户态
  |
  v
glibc拿到返回值, 返回给应用程序

```

方式二: 中断 (Interrupt)

外部硬件设备 (如键盘、网卡、定时器) 需要CPU注意时, 通过中断控制器 (APIC) 发送中断信号。CPU在执行完当前指令后, 如果 IF=1 (允许中断), 则切换到内核态处理中断。

方式三: 异常 (Exception)

CPU在执行指令过程中遇到的错误情况, 如除零、缺页 (Page Fault)、非法指令等。异常是**同步的** (由当前执行的代码引发), 而中断是**异步的** (外部事件引发)。

2.3 系统调用机制深入

x86-32: int 0x80

用户程序:

```

mov eax, 4      ; __NR_write
mov ebx, 1      ; fd = stdout
mov ecx, buf    ; const char *buf
mov edx, len    ; size_t count
int 0x80        ; 触发软中断

```

int 0x80 做了什么:

1. 从 IDT (Interrupt Descriptor Table) 查找第 0x80 号门描述符
2. 检查特权级是否允许切换 (用户态 → 内核态 OK)
3. 保存 SS, ESP, EFLAGS, CS, EIP 到内核栈
4. 加载内核的 SS, ESP
5. 跳转到中断处理函数 (system_call)

x86-64: syscall / sysret (Fast System Call)

AMD 引入了 SYSCALL / SYSRET 指令, Intel 引入了 SYSENTER / SYSEXIT。相比 int 0x80 更快的原因:

- 不经过中断描述符表 (IDT), 减少了内存访问
- 不检查栈切换细节, 使用 MSR (Model Specific Register) 预设的入口地址
- 不保存/恢复 EFLAGS (由 RFLAGS 自动保存)

- 更少的微码操作

syscall 的执行流程：

1. RCX = RIP (保存返回地址)
2. R11 = RFLAGS
3. RIP = LSTAR (MSR寄存器, 存储系统调用入口地址)
4. CS = STAR[47:32] (加载内核代码段)
5. SS = STAR[47:32] + 8 (加载内核栈段)
6. CPL = 0 (切换到 Ring 0)

sysret 的执行流程：

1. RIP = RCX
2. RFLAGS = R11
3. CS = STAR[63:48] + 16
4. CPL = 3 (回到 Ring 3)

2.4 上下文切换 (Context Switch) 的代价

上下文切换是指CPU从一个进程/线程切换到另一个进程/线程的过程。

保存和恢复的内容

每个进程的 **context** 由以下部分组成：

+-----+	
task_struct	
+-----+	
CPU寄存器快照：	
- 通用寄存器：RAX, RBX, RCX, RDX, RSI, RDI...	
- 段寄存器：CS, DS, ES, FS, GS, SS	
- 栈指针：RSP (用户栈), 内核栈指针	
- 指令指针：RIP	
- 标志寄存器：RFLAGS	
- 浮点/向量寄存器：FPU, MMX, SSE, AVX	
内存管理：	
- 页表基址 (CR3寄存器值)	
- 内存映射 (mm_struct 的 refresh)	
其他：	
- 进程状态, PID, 优先级, 调度策略	
- 打开的文件描述符, 信号处理, 命名空间	
+-----+	

上下文切换的代价

开销来源	说明
CPU寄存器保存/恢复	直接开销, 约 1-2 微秒, 需要保存约 40+ 寄存器

开销来源	说明
TLB刷新	切换进程需要切换页表 (写CR3)，导致TLB全部失效 (或需要PCID/ASID来缓解)
CPU缓存污染 (Cache Pollution)	新进程的数据和指令替换了CPU缓存中的旧内容，新进程刚开始时cache miss率高
调度器开销	CFS调度器需要维护红黑树，计算vruntime，选择下一个进程
间接开销	流水线被打断，分支预测器需要重新训练

实测数据

典型的进程上下文切换：1-10 微秒（取决于CPU架构和缓存状态）

典型的线程上下文切换：0.2-1 微秒（同一进程的线程不需要切换页表）

Linux中测量：

\$ vmstat 1 # 看 cs (context switch) 列

\$ perf stat -e cs # 统计上下文切换次数

\$ cat /proc/PID/status | grep ctxt # 某个进程的切换统计

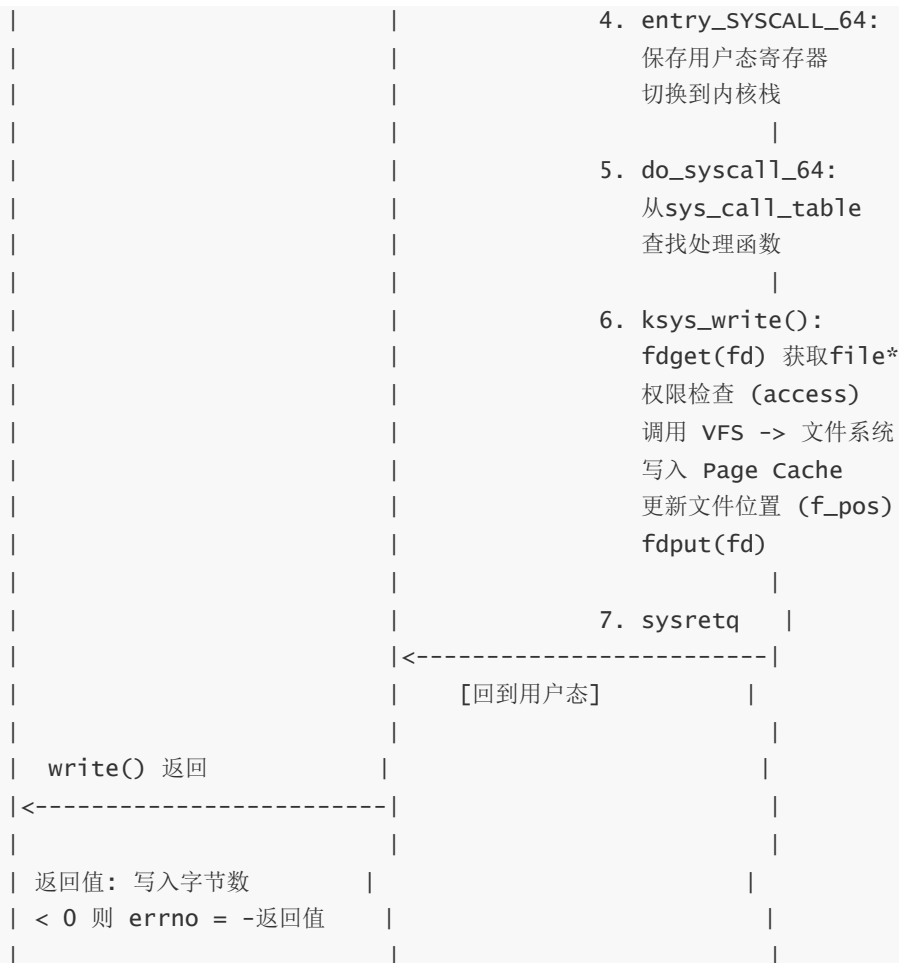
优化手段

- 减少进程/线程数量，使用事件驱动 + 非阻塞IO
- 使用协程 (coroutine) 在用户态切换，避免进入内核
- CPU亲和性绑定 (taskset / sched_setaffinity) 减少缓存失效
- 使用线程池而非频繁创建/销毁线程

3. 系统调用

3.1 系统调用端到端流程





特殊情况：如果系统调用被信号中断（EINTR）：

- glibc通常会自动重试（restart_syscall）
- 应用程序也可能收到 EINTR，需要手动处理

3.2 系统调用表与系统调用号

Linux 内核维护了一个系统调用跳转表 `sys_call_table`，以系统调用号为索引。

```

// arch/x86/entry/syscalls/syscall_64.tbl (内核源码)
// 0      common  read      __x64_sys_read
// 1      common  write     __x64_sys_write
// 2      common  open      __x64_sys_open
// 3      common  close     __x64_sys_close
// 4      common  stat      __x64_sys_newstat
// 5      common  fstat     __x64_sys_newfstat
// ...

// 调用过程简化：
long do_syscall_64(struct pt_regs *regs) {
    unsigned long nr = regs->ax; // 系统调用号
    if (nr < NR_syscalls) {
        regs->ax = sys_call_table[nr](regs); // 查表 + 调用
    }
    return regs->ax;
}

```

3.3 常见系统调用速查

分类	系统调用	功能	glibc封装
文件IO	read(0)	从fd读取	read()
	write(1)	写入fd	write()
	open(2)	打开/创建文件	open()
	close(3)	关闭fd	close()
	lseek(8)	移动文件偏移	lseek()
	stat(4)	获取文件元数据	stat()
	mmap(9)	内存映射	mmap()
	fsync(74)	刷盘	fsync()
进程控制	fork(57)	创建子进程	fork()
	vfork(58)	创建子进程(共享地址空间)	vfork()
	execve(59)	执行程序	execve()
	exit(60)	终止进程	exit()
	wait4(61)	等待子进程	waitpid()
	kill(62)	发送信号	kill()
	clone(56)	创建进程/线程	clone() (pthread用)
内存管理	brk(12)	调整数据段大小	brk() / sbrk()
	mmap(9)	映射文件或匿名内存	mmap()
	munmap(11)	解除映射	munmap()
	mprotect(10)	修改内存保护属性	mprotect()
网络	socket(41)	创建套接字	socket()
	connect(42)	连接	connect()
	bind(49)	绑定地址	bind()
	listen(50)	监听	listen()
	accept(43)	接受连接	accept()
	sendto(44), recvfrom(45)	收发UDP数据	sendto/recvfrom
其他	getpid(39)	获取PID	getpid()
	nanosleep(35)	休眠	nanosleep()

分类	系统调用	功能	glibc封装
	epoll_create/ctl/wait	IO多路复用	epoll_*

3.4 strace 实战演示

```
# 1. 追踪一个最简单的程序
$ strace echo "hello"
execve("/usr/bin/echo", ["echo", "hello"], ...) = 0
brk(NULL) = 0x55555576b000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8c0e2a5000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF...", 832) = 832
fstat(3, {st_mode=S_IFREG|0755, ...}) = 0
mmap(NULL, 1857472, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f8c0e0de000
mprotect(0x7f8c0e103000, 1671168, PROT_NONE) = 0
...
write(1, "hello\n", 6) = 6
close(1) = 0
close(2) = 0
exit_group(0) = ?

# 2. 追踪特定系统调用
$ strace -e trace=open,openat,read,write ls
# 只看文件操作

# 3. 追踪运行中的进程
$ strace -p PID

# 4. 统计系统调用次数
$ strace -c ls
% time      seconds    usecs/call     calls    errors syscall
-----
24.53      0.000150         15         10         0    mmap
20.93      0.000128         10         13         0    mprotect
11.77      0.000072          7         10         0    openat
10.62      0.000065          5         12         0    close
 9.80      0.000060          7          8         0    fstat
...

# 5. 显示时间戳
$ strace -tt -T ls
# -tt: 微秒级时间戳
# -T: 每次调用的耗时

# 6. 追踪子进程
$ strace -f ./a.out

# 7. 只看失败的调用
$ strace -z ./a.out
```

4. 进程管理

4.1 进程的创建 — fork() 深度剖析

`fork()` 是Linux中创建新进程的唯一方式 (`clone`是底层实现)。`fork`调用一次，返回两次：

- 在父进程中返回子进程的PID (> 0)
- 在子进程中返回 0
- 失败返回 -1 (如进程数达到上限)

fork() 的完整流程

用户进程调用 `fork()`

|

v

glibc: 设置 `eax = __NR_fork (57)`, `syscall`

|

v

`[_do_fork() / kernel_clone()]` (内核 5.x+)

|

v

1. 分配新的 PID

- `alloc_pid()` 从pid位图中分配

|

v

2. 复制 `task_struct` (通过 `copy_process()`)

++- `copy_files()` - 复制文件描述符表 (`fdtable`)

| 默认: 父子共享文件描述符, 引用计数+1

| (`fork`后父子操作同一个fd会影响彼此的文件偏移)

++- `copy_fs()` - 复制文件系统信息 (`fs_struct`)

| 共享 `umask`, 根目录, 当前工作目录 (`pwd`)

++- `copy_sighand()` - 复制信号处理表

| 信号处理函数 (`sigaction`) 复制

| 挂起的信号队列清空 (子进程不应收到父进程的信号)

++- `copy_mm()` - 复制内存管理结构 (`mm_struct`) ★核心★

| [Copy-On-Write 机制, 详见第8节]

| 不真正复制物理页, 而是:

| - 复制页表, 共享物理页

| - 将父子进程的页表项都标记为只读

| - 设置 `COW` 标志

++- `copy_namespaces()` - 复制命名空间

++- `copy_thread()` - 复制线程/寄存器上下文

| 设置子进程的返回值为 0

| (通过修改内核栈上的 `pt_regs->ax`)

++- ...

|

v

3. 设置子进程状态为 `TASK_RUNNING`, 加入调度队列

|

v

4. `fork` 返回

- 父进程: 返回子进程PID

- 子进程：返回0（通过 `copy_thread` 中设置的 `pt_regs->ax=0`）

注意：`fork` 后父子进程的执行顺序不确定（取决于调度器）
两者是独立的进程，各自有自己的地址空间（COW后）

fork + exec 组合

这是创建新程序的标准模式：

```
pid_t pid = fork();
if (pid == 0) {
    // 子进程
    // close inherited fds, set env, redirect stdin/stdout...
    execve("/bin/ls", argv, envp);
    // execve成功则不会返回；失败则：
    perror("execve");
    _exit(127);
} else if (pid > 0) {
    // 父进程
    int status;
    waitpid(pid, &status, 0);
}
```

vfork() — 被 fork+COW 替代

`vfork()` 的历史意义：在没有COW的年代，`vfork()` 保证子进程先运行且共享父进程地址空间，用于 `fork + exec` 场景避免无用的地址空间复制。有了COW后，`fork()` 同样高效，`vfork()` 基本被废弃，仅保留用于兼容。

4.2 exec() 家族

`exec` 系列函数**不创建新进程**，而是用新程序**替换**当前进程的地址空间。PID不变，但代码段、数据段、堆、栈全部被替换。

`execve()` 执行流程：

```
用户调用 execve("/bin/ls", argv, envp)
    |
    v
sys_execve()
    |
    v
do_execveat_common()
    |
    v
1. 打开可执行文件
   - open_exec() 打开文件
     |
     v
2. 加载 ELF 头
   - kernel_read() 读取 ELF header
   - 验证 e_ident (魔数: 0x7f 'E' 'L' 'F')
   - 读取 program headers (PT_LOAD段)
```

```

|
v
3. 替换地址空间
- flush_old_exec() 清理旧的 mm_struct
- 建立新的 mm_struct
|
v
4. 加载段 (load_elf_binary)
+-- 代码段 (.text): PROT_READ | PROT_EXEC, MAP_PRIVATE
+-- 数据段 (.data, .bss): PROT_READ | PROT_WRITE, MAP_PRIVATE
+-- 动态链接器 (ld.so): 如果是动态链接的程序, 先加载 ld.so
+-- 栈: 建立用户栈, 放入 argv, envp, auxv
|
v
5. 设置用户态入口
- 静态链接: ELF entry point (e_entry)
- 动态链接: ld.so 的入口
|
v
6. 返回用户态, 从新的入口开始执行

```

注意: `execve` 后:

- PID 不变
- 设置了 `FD_CLOEXEC` 的文件描述符自动关闭
- 信号处理恢复为默认 (`SIG_DFL`)
- 有效权限可能改变 (如果文件有 `setuid/setgid` 位)

exec 家族函数:

函数	参数格式	路径搜索
<code>execve(path, argv[], envp[])</code>	数组 + 完整路径	不搜索PATH
<code>execle(path, arg0, ..., NULL, envp[])</code>	可变参数 + 完整路径	不搜索PATH
<code>execlp(file, arg0, ..., NULL)</code>	可变参数	搜索PATH
<code>execvp(file, argv[])</code>	数组	搜索PATH
<code>execvpe(path, argv[], envp[])</code>	数组 + 完整路径	搜索PATH

4.3 PCB — task_struct 详解

Linux 中的进程控制块 (PCB) 就是 `task_struct` 结构体, 通常占据约 2-8KB。

```

// include/linux/sched.h 中的重要字段 (简化描述)

struct task_struct {
    // === 调度相关 ===
    volatile long state;           // 进程状态: TASK_RUNNING, TASK_INTERRUPTIBLE等
    int prio;                      // 动态优先级
    int static_prio;               // 静态优先级 (nice值影响)

```

```

int normal_prio;           // 归一化优先级
unsigned int rt_priority;  // 实时优先级
struct sched_entity se;    // CFS调度实体 (包含vruntime)
struct sched_rt_entity rt; // 实时调度实体
unsigned int policy;       // 调度策略: SCHED_NORMAL, SCHED_FIFO, SCHED_RR

// === 进程标识 ===
pid_t pid;                 // 进程ID
pid_t tgid;                // 线程组ID (getpid()实际返回这个)
struct task_struct *group_leader; // 线程组leader
struct list_head thread_group; // 线程组链表

// === 亲属关系 ===
struct task_struct *real_parent; // 真正的父进程
struct task_struct *parent;      // 父进程 (ptrace时可能是tracer)
struct list_head children;       // 子进程链表
struct list_head sibling;        // 兄弟进程链表

// === 地址空间 ===
struct mm_struct *mm;           // 内存描述符
struct mm_struct *active_mm;    // 实际使用的mm (内核线程mm=NULL时用)

// === 文件 ===
struct files_struct *files;     // 文件描述符表
struct fs_struct *fs;          // 文件系统信息 (root, pwd, umask)

// === 信号 ===
struct signal_struct *signal;   // 信号描述符 (线程组共享)
struct sighand_struct *sighand; // 信号处理表 (线程组共享)
sigset_t blocked;              // 阻塞信号集
struct sigpending pending;      // 私有挂起信号

// === 上下文 ===
struct thread_struct thread;    // 线程上下文 (寄存器快照)
// 包含: sp (栈指针), ip (指令指针), fsbase, gsbase

// === 栈 ===
void *stack;                   // 进程内核栈 (thread_info 通常嵌在栈底)

// === 其他 ===
struct list_head tasks;        // 全局进程链表节点
unsigned long nvcsw;            // 自愿上下文切换次数
unsigned long nivcsw;          // 非自愿上下文切换次数
u64 start_time;                // 进程启动时间 (ns)
u64 real_start_time;           // 进程实际启动时间
char comm[TASK_COMM_LEN];      // 进程名 (最多16字节)
struct cred *cred;              // 权限凭证 (uid, gid, cap...)
cpumask_t cpus_mask;           // CPU亲和性
// ... 还有很多字段

```

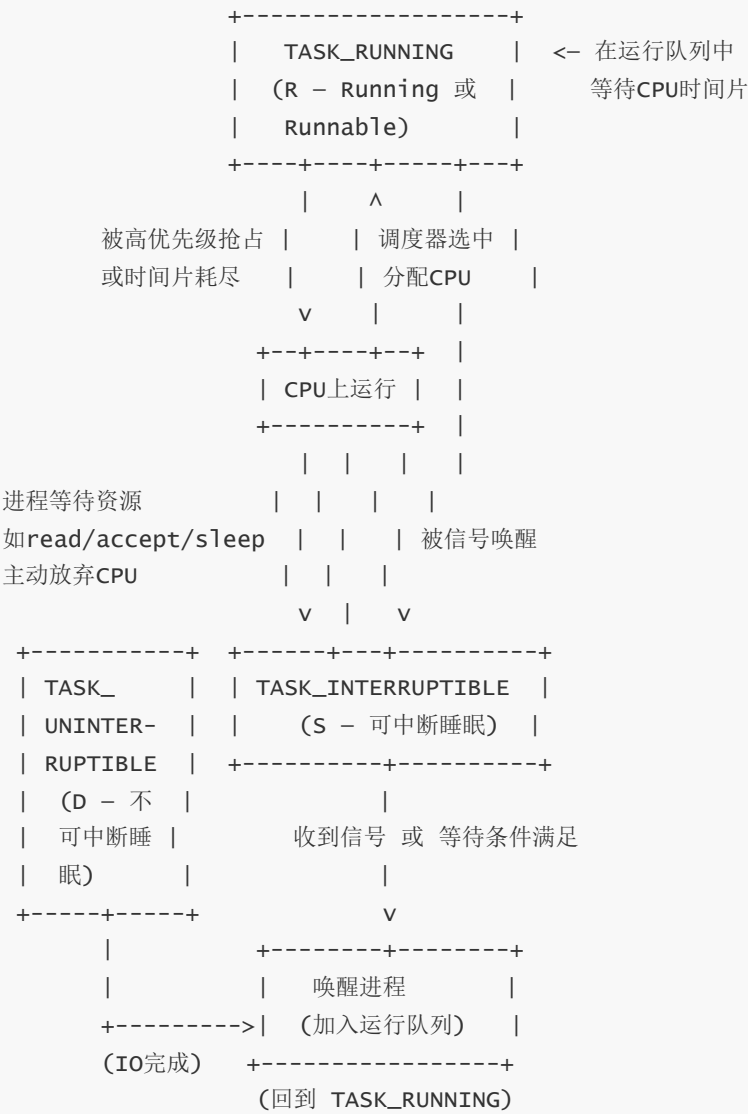
```
};
```

面试要点: task_struct 不是通过堆分配 (kmalloc) 的, 而是通过内核栈隐式分配。在 x86-64 Linux 中, 内核栈大小为 `THREAD_SIZE` (通常 16KB = 4 pages), 栈底放置 `thread_info` (在较新内核中简化为 task_struct 指针), task_struct 通过 slab 分配器分配。

```
// 获取当前进程的 task_struct (高效)
#define current get_current()
// 实际上是通过内核栈找到 thread_info, 再找到 task_struct
// 或者通过 percpu 变量直接获取
```

4.4 进程状态与转换

进程状态转移图



- 其他状态:
- TASK_STOPPED (T): 收到 SIGSTOP/SIGTSTP, 暂停
 - TASK_TRACED (t): 正在被 ptrace (gdb调试)
 - EXIT_ZOMBIE (Z): 进程已退出, 父进程尚未回收 (wait)
 - EXIT_DEAD (X): 进程已完全清理

用户看到的 (ps/top):
R - Running or Runnable (运行或就绪)

S - Sleeping (可中断睡眠)
D - Disk Sleep (不可中断睡眠, 通常在等待IO)
T - Stopped (暂停)
Z - Zombie (僵尸)
X - Dead (死亡, 不应可见)

状态详解:

状态	标志	含义	接收信号	示例
TASK_RUNNING	0x0000	正在运行或在运行队列中等待CPU	—	正常执行的程序
TASK_INTERRUPTIBLE	0x0001	可中断的睡眠, 等待某条件满足	是	<code>read()</code> 等待键盘输入
TASK_UNINTERRUPTIBLE	0x0002	不可中断的睡眠, 通常等IO	否	<code>read()</code> 等待磁盘IO
TASK_STOPPED	0x0008	收到停止信号, 暂停执行	—	按Ctrl-Z
EXIT_ZOMBIE	0x0010	进程已退出, 等待父进程wait()	否	子进程退出后父进程未回收
EXIT_DEAD	0x0020	父进程已wait(), task_struct可释放	否	最终状态

4.5 僵尸进程 (Zombie) 与孤儿进程 (Orphan)

僵尸进程

定义: 子进程先于父进程退出, 但父进程没有调用 `wait()` / `waitpid()` 来回收子进程的退出状态。此时子进程的 `task_struct` 和少量资源仍然保留 (主要是退出码、资源使用统计等), 但进程本身已经不再执行。

```
产生僵尸进程的示例:
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // 子进程: 立即退出
        exit(0);
    } else {
        // 父进程: 一直运行, 不wait
        while(1) sleep(1);
    }
    return 0;
}
// 运行后 ps aux 会看到 [a.out] <defunct> 或 Z 状态
```

危害:

- 占用 PID (PID 资源有限, 默认最大 32768)
- 占用少量内核内存 (task_struct 仍需保留)

- 大量僵尸进程可能导致 `fork()` 失败 (ENOMEM — 无法分配PID)

解决方法:

```
// 方法1: 父进程调用 wait() / waitpid()
wait(NULL);           // 等待任意子进程
waitpid(pid, &status, 0); // 等待特定子进程
waitpid(-1, &status, WNOHANG); // 非阻塞等待

// 方法2: 忽略 SIGCHLD 信号 (系统自动回收)
signal(SIGCHLD, SIG_IGN);

// 方法3: 使用 SA_NOCLDWAIT
struct sigaction sa;
sa.sa_handler = SIG_DFL;
sa.sa_flags = SA_NOCLDWAIT;
sigemptyset(&sa.sa_mask);
sigaction(SIGCHLD, &sa, NULL);

// 方法4: 双重fork (让init进程成为祖先进程的父进程)
if (fork() == 0) {
    if (fork() == 0) {
        // 孙子进程: 被init "收养", init会自动wait
        // ... 执行实际工作
        exit(0);
    }
    exit(0); // 子进程立即退出
}
wait(NULL); // 父进程回收子进程, 孙子进程成为孤儿由init接管
```

孤儿进程

定义: 父进程先于子进程退出, 子进程变为"孤儿", 由 init 进程 (PID=1) 或最近的 subreaper 进程 "收养"。孤儿进程退出后, init/subreaper 会自动调用 `wait()` 回收, **不会产生僵尸进程**。

```
// 孤儿进程示例
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        sleep(10); // 子进程活得比父进程久
        printf("My parent now is PID=%d\n", getppid()); // 将输出 1
        exit(0);
    } else {
        sleep(1); // 父进程先退出
        exit(0);
    }
}
```

4.6 进程调度

CFS (Completely Fair Scheduler) — Linux 默认调度器

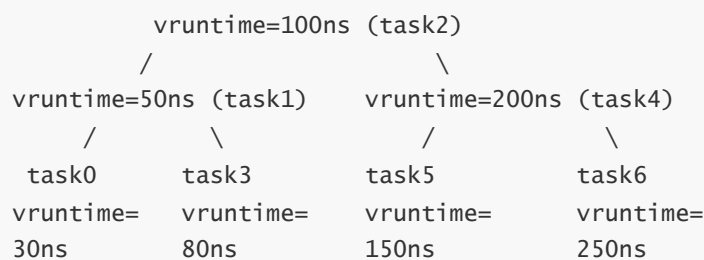
Linux 2.6.23 引入，目标是公平地将CPU时间分配给所有进程。

核心概念: vruntime (虚拟运行时间)

```
// vruntime 计算公式 (简化):  
vruntime = real_runtime * (NICE_0_LOAD / weight)  
// 其中 weight 由 nice 值决定  
  
// nice 值越低 (优先级越高), weight 越大  
// weight 大 -> vruntime 增长慢 -> 进程获得更多CPU时间  
  
// nice 值映射到 weight (近似):  
nice:  -20  ...   0   ...  +19  
weight: 88761 ... 1024 ... 15
```

核心数据结构: 红黑树 (rbtree)

CFS 运行队列 (cfs_rq) 维护一棵红黑树:



CFS 总是选择最左边的节点 (vruntime 最小) 来运行
红黑树插入/删除复杂度: $O(\log n)$

调度流程:

每个tick或事件触发时:

1. `update_curr()`: 更新当前进程的 `vruntime`
`vruntime += delta_exec * (NICE_0_LOAD / se->load.weight)`
2. 检查是否需要抢占:
 - 如果有更高优先级的进程加入
 - 如果有进程的 `vruntime` 比当前进程小足够多
(`sched_min_granularity`: 最小调度粒度, 典型 0.75ms)
3. 如果需要切换:
 - a. 当前进程放回红黑树
 - b. 从红黑树取最左节点 (`pick_next_task_cfs()`)
 - c. `context_switch()` 执行上下文切换

调度周期 (`sched_period`):

- `nr_running <= 8`: `period = 6ms` (默认 `sched_latency_ns = 6ms`)
- `nr_running > 8`: `period = nr_running * 0.75ms (sched_min_granularity)`

调度策略:

策略	类型	优先级范围	说明
SCHED_OTHER (SCHED_NORMAL)	CFS	nice -20~+19	默认，普通进程
SCHED_BATCH	CFS	nice -20~+19	批处理，不抢占
SCHED_IDLE	CFS	最低	极低优先级
SCHED_FIFO	RT	1-99	实时，先入先出，可能饿死
SCHED_RR	RT	1-99	实时，轮转，同优先级时间片轮转
SCHED_DEADLINE	DL	—	最新，每个任务有截止时间保证

5. 线程

5.1 线程 vs 进程 对比表

维度	进程 (Process)	线程 (Thread)
定义	资源分配的基本单位	CPU调度的基本单位
PID	每个进程有独立PID	线程有独立TID，但同一进程的线程共享TGID (即PID)
地址空间	独立的虚拟地址空间	共享所属进程的地址空间
文件描述符	独立的fd表	共享fd表 (fd是进程级别资源)
信号处理	独立	共享 (信号发给进程，任意一个线程处理)
堆空间	独立	共享 (malloc/free 是线程安全的)
栈空间	独立	各自独立 (每个线程有自己的栈)
创建开销	大 (需要复制mm_struct等)	小 (共享大部分资源)
上下文切换	慢 (需要切换页表，刷新TLB)	快 (同一进程线程切换不切换页表)
IPC通信	需要IPC机制 (管道、消息队列、共享内存等)	简单 (直接通过共享变量)
崩溃影响	不影响其他进程	通常导致整个进程崩溃
Linux内核视角	task_struct	也是 task_struct (轻量级进程)
创建系统调用	clone(SIGCHLD, ...)	clone(CLONE_VM CLONE_FS CLONE_FILES CLONE_SIGHAND, ...)

5.2 Linux 线程实现 — clone() 系统调用

Linux 内核中**没有独立的线程概念**。线程和进程在底层都是 `task_struct`，区别在于 `clone()` 时共享哪些资源。

```
// Linux中创建进程 (fork)
clone(SIGCHLD, 0); // 共享：只有信号处理

// Linux中创建线程 (pthread_create 底层)
clone(CLONE_VM |           // 共享地址空间
      CLONE_FS |           // 共享文件系统信息 (root, pwd)
      CLONE_FILES |        // 共享文件描述符表
      CLONE_SIGHAND |       // 共享信号处理表
      CLONE_THREAD |        // 放入同一个线程组
      CLONE_SYSVSEM |       // 共享System V信号量
      CLONE_SETTLS,         // 设置线程局部存储
      0);

// 关键 CLONE_* 标志位：
CLONE_VM      - 共享mm_struct (共享地址空间)
CLONE_FS      - 共享fs_struct (共享 cwd, root, umask)
CLONE_FILES   - 共享files_struct (共享 fd表)
CLONE_SIGHAND - 共享sighand_struct (共享信号处理)
CLONE_THREAD  - 线程组标识 (设置 tgid)
CLONE_PARENT  - 子进程的父进程与调用者的父进程相同
CLONE_VFORK   - 父进程阻塞直到子进程退出或exec
CLONE_NEWNS   - 新的mount命名空间
CLONE_NEWPID  - 新的PID命名空间
CLONE_NEWNET  - 新的网络命名空间 (容器的基础)
```

线程在/proc中的呈现:

```
$ ls /proc/PID/task/      # 查看进程的所有线程
1000/ 1001/ 1002/

# 每个线程目录下有各自的：
/proc/PID/task/TID/status  # 线程信息
/proc/PID/task/TID/sched   # 调度统计
/proc/PID/task/TID/stack   # 内核栈
```

为什么 Linux 用 task_struct 统一表示进程和线程？

这是 Linux 的哲学胜利——"一切皆任务"。通过灵活的组合 clone 标志，可以用统一的机制实现进程、线程、容器。这比 Windows 那种进程/线程是完全不同数据结构的做法更简洁。

5.3 内核线程 vs 用户线程

维度	用户线程 (User Thread)	内核线程 (Kernel Thread)
创建者	应用程序 (通过pthread)	内核模块或内核自身
地址空间	使用用户地址空间 (有mm)	仅使用内核地址空间 (mm = NULL)

维度	用户线程 (User Thread)	内核线程 (Kernel Thread)
可见性	<code>ps -eLf</code> 可见	<code>ps -ef</code> 可见, 名称通常加 <code>[]</code>
示例	普通线程	<code>[kworker/0:0]</code> , <code>[ksoftirqd/0]</code> , <code>[migration/0]</code>
创建方式	<code>pthread_create()</code> -> <code>clone()</code>	<code>kthread_create()</code> -> <code>wake_up_process()</code>
调度	同普通进程	仅在内核态运行, 不被换出

常见内核线程:

<code>[kthreadd]</code>	- <code>PID=2</code> , 所有内核线程的父线程
<code>[ksoftirqd/N]</code>	- 每CPU的软中断处理线程
<code>[kworker/N:M]</code>	- 工作队列线程
<code>[migration/N]</code>	- 负载均衡迁移线程
<code>[watchdog/N]</code>	- 软锁检测线程
<code>[rcu_sched]</code>	- RCU (Read-Copy-Update) 线程

5.4 pthread 基础

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

// 线程函数
void *thread_func(void *arg) {
    int *val = (int *)arg;
    printf("Thread: val=%d\n", *val);
    return (void *) (intptr_t) (*val * 2);
}

int main() {
    pthread_t tid;
    int arg = 42;

    // 1. 创建线程
    if (pthread_create(&tid, NULL, thread_func, &arg) != 0) {
        perror("pthread_create");
        exit(1);
    }

    // 2. 等待线程结束 + 获取返回值
    void *retval;
    pthread_join(tid, &retval);
    printf("Thread returned: %ld\n", (long)retval);

    // 3. 分离线程 (不需要join, 资源自动回收)
    // pthread_detach(tid);

    // 4. 线程取消
    // pthread_cancel(tid);
}
```

```
// 5. 线程退出（在线程函数中调用）
// pthread_exit(retval);

return 0;
}
// 编译：gcc -pthread thread_demo.c -o thread_demo
```

pthread 常见问题:

问题	说明
joinable vs detached	joinable线程需要pthread_join回收，detached线程退出时自动回收。默认joinable。
线程栈大小	默认约 8MB (ulimit -s)。可设 pthread_attr_setstacksize()。大量线程需调小。
线程安全	标准库函数分为线程安全(带 _r 后缀如 strtok_r)和非线程安全。
互斥锁	pthread_mutex_t 用于保护临界区
条件变量	pthread_cond_t 用于线程间通知等待
线程局部存储	__thread 关键字，NPTL通过 %fs 寄存器实现

6. 内存管理

6.1 虚拟内存 (Virtual Memory) 深度剖析

为什么需要虚拟内存?

虚拟内存的四大作用：

1. 隔离 (Isolation)
每个进程有独立的地址空间，进程A不能访问进程B的内存。
通过页表中的权限位，MMU在硬件层面强制隔离。
2. 抽象 (Abstraction)
每个进程"以为"自己独占全部内存（32位系统可寻址4GB）。
实际物理内存可能比地址空间小（通过磁盘交换）或大。
3. 共享 (Sharing)
多个进程可以映射同一物理页：
 - 共享库（libc.so 只加载一份物理拷贝）
 - 共享内存（SHM）
 - Copy-On-Write（父子进程）
4. 内存过量分配 (Overcommit)
进程可以分配比物理内存更多的虚拟内存（取决于内核 overcommit 策略）。
只有在实际使用时才分配物理页（demand paging）。

虚拟地址到物理地址的映射

32位系统经典划分（32-bit，4GB虚拟地址空间）：

+-----+ 0xFFFFFFFF (4GB)	
KERNEL	
SPACE	1GB (0xC0000000 - 0xFFFFFFFF)
(每个进程相同)	
+-----+ 0xC0000000 (3GB)	
USER	3GB (0x00000000 - 0xBFFFFFFF)
SPACE	
(每个进程独立)	
+-----+	
Stack	向下增长
v	
^	
Heap	(通过 brk/sbrk 扩展)

.bss	未初始化数据
.data	已初始化数据
.text	只读代码
+-----+	
+-----+ 0x00000000	

64位：虚拟地址空间巨大（48位实际使用 = 256TB，
高128TB内核，低128TB用户）
/proc/sys/vm/mmap_min_addr 限制了最小映射地址（防NULL指针漏洞）

6.2 分页 (Pagination) 机制

物理内存被划分为固定大小的**页框** (Page Frame)，虚拟内存划分为同样大小的**页** (Page)。

- x86常见页大小: 4KB (标准页), 2MB (大页/huge page), 1GB (超大页)
- ARM64也支持 4KB, 64KB, 2MB, 1GB

虚拟地址结构（x86-64，4级页表，48位虚拟地址，4KB页）：

63	48	47	39	38	30	29	21	20	12	11	0
+-----+-----+-----+-----+-----+-----+											
Sign		PML4		PDPT		PD		PT		Offset	
Ext.		(9 bits)		(9 bits)		(9 bits)		(9 bits)		(12 bits)	
+-----+-----+-----+-----+-----+-----+											
^		^		^		^		^		^	
										+-- 页内偏移（4KB页）	
								+-- Page Table（页表）索引			
				+-- Page Directory（页目录）索引							
				+-- Page Directory Pointer Table 索引							
		+-- Page Map Level 4 索引									

页表项 (PTE — Page Table Entry) 结构:

x86-64 PTE (64 bits), 4KB页:

```

 63   62-52   51-12   11-9  8  7  6  5  4  3  2  1  0
+---+-----+-----+-----+---+---+---+---+---+---+---+---+
|XD| (res)  | PFN   |(ign)|G|0|D|A|PCD|PWT|U/S|R/W|P|
+---+-----+-----+-----+---+---+---+---+---+---+---+---+

```

P (Present) – 页是否在物理内存中（不在则触发缺页）
R/W (Read/Write) – 0=只读, 1=可写（COW时设为只读）
U/S (User/Super) – 0=内核, 1=用户可访问
PWT, PCD – 缓存控制
A (Accessed) – 页是否被访问过（由MMU设置）
D (Dirty) – 页是否被写过（由MMU设置）
G (Global) – CR3切换时不刷新此TLB项
XD (eXecute Disable) – 禁止执行（NX bit, 防缓冲区溢出）
PFN – 物理页帧号（Physical Frame Number）

6.3 多级页表 (Multi-Level Page Table)

多级页表地址翻译过程（x86-64, 4级）:

虚拟地址: 0x00007F A1B2 C000

划分:

PML4[0] -> PDPT[255] -> PD[0x81] -> PT[0x12C] + offset[0x000]

翻译流程:

CR3（页表基址寄存器）

```

|
v
+-----+
| PML4 | 第4级页表
| 表   |
|      | 每个进程有自己的 PML4 物理地址（CR3指向）
+---+---+
|
| PML4[0] -> 找到对应的 PDPT 地址
v
+-----+
| PDPT | 页目录指针表
| 表   |
+---+---+
|
| PDPT[255] -> 找到对应的 PD 地址
v
+-----+
| PD   | 页目录

```

```

| 表 |
+---+-----+
|
| PD[0x81] -> 找到对应的 PT 地址
v
+-----+
| PT | 页表（最后一级）
| 表 |
+---+-----+
|
| PT[0x12C] -> 得到 PTE: PFN=0x12345, flags=RW+USER+PRESENT
v
+-----+
| 物理页框 | 0x12345000（4KB对齐的物理地址）
|
+-----+
|
| offset[0x000] -> 最终物理地址 = 0x12345000 + 0x000 = 0x12345000
v
物理地址：0x12345000

```

总共需要：4次内存访问（4级页表查询）+ 1次数据访问 = 5次内存访问
这就是为什么需要 TLB！（见 6.4）

为什么需要多级页表？

单级页表的问题: 对于 64位系统，4KB页，单级页表需要 2^{52} 个表项 (不可思议的巨大)。多级页表的优势:

1. **节省内存**: 只分配实际使用的中间页表
2. **灵活性**: 大页 (2MB/1GB) 可以在中间级别直接映射，不需最后一层

6.4 TLB (Translation Lookaside Buffer)

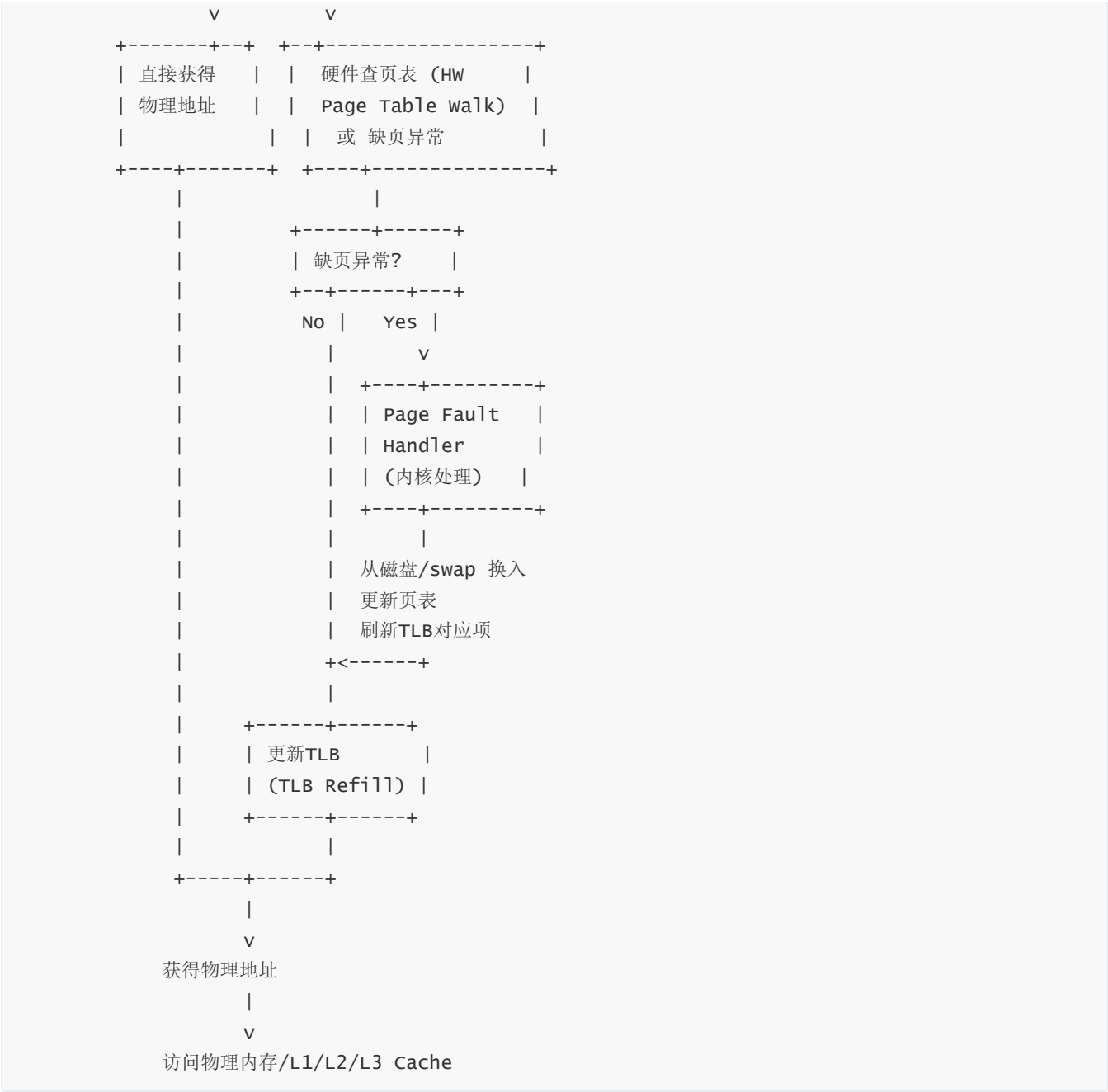
TLB 是 MMU 内部的高速缓存，用于缓存最近使用的虚拟地址到物理地址的翻译结果。这是让虚拟内存性能可接受的关键硬件结构。

CPU访问内存时的 MMU 翻译流程：

```

CPU发出虚拟地址
|
v
+-----+
| 查 TLB | <-- 硬件缓存，通常很小
| (L1 TLB: | L1 TLB: ~64 entries (数据)
| 4-cycle | L2 TLB: ~1500 entries (统一)
| latency) |
+-----+-----+
|
+-----+-----+
| TLB Hit? |
+-----+-----+
Yes | No (TLB Miss)
|

```



TLB 的技术细节:

属性	典型值	说明
L1 I-TLB	64-128 entries, 4-way, 4-cycle	指令TLB
L1 D-TLB	32-64 entries, 4-way, 4-cycle	数据TLB
L2 STLB	512-1536 entries, 8-way, ~12-cycle	大页TLB也在此
TLB flush	写CR3时全部刷新	但Global页不刷新 (PGE=1)
PCID	每个进程一个ID (12bit)	避免切换进程时刷新TLB
ASID	ARM版的PCID	同样用途

TLB 友好编程:

- 使用大页 (2MB/1GB) 减少TLB miss (一个2MB页覆盖512个4KB页)
- 尽量顺序访问，保持空间局部性
- 避免过多小范围的随机内存访问

6.5 缺页异常 (Page Fault) 处理流程

缺页异常处理完整流程

用户访问一个虚拟地址 (如: `*ptr = 42`)

|

v

```
+-----+
| MMU查页表:      |
| PTE.Present = 0 |
| (页不在物理内存) |
+-----+
```

|

v

```
+-----+
| CPU触发 #PF 异常 |
| 保存现场          |
| 切换到内核态      |
+-----+
```

|

v

```
+-----+
| do_page_fault() |
| (arch/x86/mm/    |
| fault.c)         |
+-----+
```

|

v

```
+----+ 检查地址合法性  +----+
|    | (用户空间? 已映射?) |    |
```

```
+----+-----+-----+
合法 |                               | 不合法
```

v

v

```
+-----+ +-----+
| 什么类型? | | SIGSEGV |
+-----+ | (Segfault) |
```

|

+-----+

```
+-----+
```

|

v

|

v

```
+-----+ +-----+
| Minor Fault| | Major Fault |
| (小缺页)   | | (大缺页)   |
|           | |           |
| 原因:      | | 原因:      |
| 页被释放但 | | 页被换出到磁盘 |
| 还在内存中 | | (swap) 或   |
| (如cow后)  | | mmap的文件页 |
```

	不在page cache
处理：	处理：
1. 分配新页	1. 分配新页
2. 复制数据 (若是COW)	2. 从磁盘读入 (block IO)
3. 更新页表	3. 更新页表
开销：微秒级	开销：毫秒级


```

+-----+-----+ +-----+-----+
|                                     |
|                                     |
+-----+-----+
|                                     |
|                                     |
v
+-----+-----+
| 更新页表PTE:                      |
| P = 1, 设置权限                    |
| 可能需要刷新TLB                    |
+-----+-----+
|                                     |
|                                     |
v
+-----+-----+
| iret / sysret                      |
| 回到用户态                        |
| 重新执行失败指令                  |
+-----+-----+
|                                     |
|                                     |
v
+-----+-----+
| 这次MMU查页表                      |
| PTE.Present = 1                    |
| 正常翻译, 继续执行                |
+-----+-----+

```

区分 Minor 和 Major Fault:

```

# 查看进程的缺页统计
$ cat /proc/PID/stat
# 字段:
# minflt - minor page faults (不需要磁盘IO)
# majflt - major page faults (需要磁盘IO)

# 实时监控缺页
$ perf stat -e page-faults ./a.out
$ perf stat -e minor-faults,major-faults ./a.out

# strace 中看不出来 (缺页是异常, 不是系统调用)

```

7. malloc/free

7.1 malloc 内部工作原理

malloc() 并不是每次都调用 brk() 或 mmap() (那样太慢了), 它维护了内存池进行内存管理。

glibc ptmalloc 设计概览

ptmalloc (glibc的内存分配器) 结构:

+-----+

|
| ptmalloc 核心组件
|

+-----+

|
|
| 1. Arena (分配区)
| - Main Arena: 主分配区 (通过brk扩展)
| - Thread Arena: 线程分配区 (通过mmap)
| - 每个线程尽量固定一个Arena(减少锁竞争)
|
| 2. Chunk (内存块)
| [prev_size][size][fd][bk][...data...]
| - in-use chunk
| - free chunk (加入bin)
|
| 3. Bin (空闲链表/容器)
| - Fast Bins: 16~80字节, LIFO, 单链表
| - Small Bins: 16~1008字节, FIFO, 双链表
| - Large Bins: >1008字节, 按大小排序
| - Unsorted Bin: 最近释放的chunk暂存区
|
| 4. Top Chunk
| Arena顶部的剩余空间, 所有bin都无合适
| 大小的chunk时从这里切
|
|

+-----+

malloc 分配流程

用户调用 malloc(100)

|

v

请求大小调整:

实际大小 = max(100, MINSIZE) + overhead = 对齐到 16/8 字节边界

例如: 100 -> 112 (加头部16字节, 对齐到16的倍数)

|

v

+--> 检查 Fast Bin 中 有无合适大小的chunk

| (大小 <= global_max_fast, 默认 128字节)

| |

| |-- 有: 取出 (LIFO, 从链表头取) --> 返回

| |

| v 无

```

|  检查 Small Bin 中 有无合适大小的chunk
|  (大小 <= 1008 字节)
|  |
|  |-- 有: 取出 (FIFO, 取第一个合适的) --> 返回
|  |
|  v  无
|  检查 Unsorted Bin
|  遍历 unsorted bin 中的chunk:
|  - 如果大小完全匹配: 直接返回
|  - 否则: 分类放入 small bin 或 large bin
|  - 如果在此过程中找到了合适的, 返回
|  |
|  v  仍无
|  检查 Large Bin
|  (大小 > 1008 字节)
|  寻找 "best fit" (最接近且不小于请求大小的chunk)
|  |
|  |-- 有: 取出, 可能分割 (多余部分放回) --> 返回
|  |
|  v  无
|  尝试从 Top Chunk 切一块
|  |
|  |-- Top Chunk 足够: 切分, 更新top --> 返回
|  |
|  v  Top Chunk 不够
|  调用 sbrk() 或 mmap() 扩展堆
|  |
|  +-- 大小 >= MMAP_THRESHOLD (默认 128KB): 用 mmap 分配
|  +-- 大小 < MMAP_THRESHOLD: 用 sbrk/brk 扩展heap
|  |
|  v  仍失败
|  返回 NULL (设置 errno = ENOMEM)

```

7.2 brk() vs mmap()

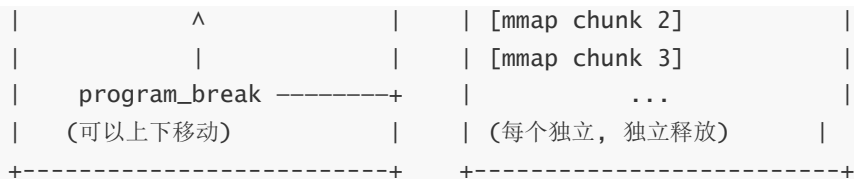
brk() / sbrk():

- 扩展进程的数据段 (heap)
- 连续的内存, Heap顶端增长
- 释放后通常不能立即归还OS (可能导致内存碎片保留)
- 适合小内存分配 (< 128KB 默认)
- 所有brk分配的内存存在同一个连续区间

mmap() (MAP_ANONYMOUS | MAP_PRIVATE):

- 在进程地址空间中找到一段空闲区域
- 可以独立释放 (munmap), 内存立刻归还OS
- 适合大内存分配 (>= 128KB 默认)
- 每块独立, 不会产生外部碎片 (但浪费虚拟地址空间)

+-----+	+-----+
brk heap	mmap region
+-----+	+-----+
[chunk1][chunk2][FREE]...	[mmap chunk 1]



mmap 阈值调优:

```
# 查看当前 mmap 阈值 (字节)
$ cat /proc/sys/vm/mmap_min_addr # 最小映射地址

# mallopt 控制阈值
#include <malloc.h>
mallopt(M_MMAP_THRESHOLD, 128*1024); // 设置阈值为128KB
mallopt(M_MMAP_MAX, 65536);          // 最大mmap区域数
```

7.3 ptmalloc (glibc) 内部结构

chunk 结构

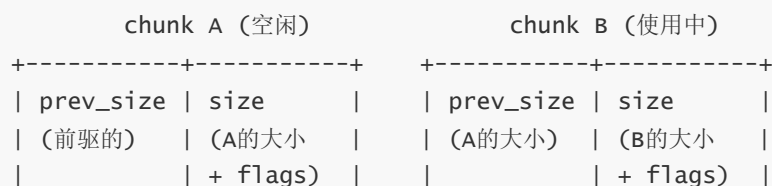
```
// 正在使用的 chunk
struct malloc_chunk {
    size_t prev_size; // 前一个chunk的大小 (如果前一个是空闲的)
    size_t size;      // 当前chunk大小 (低3位是标志位)
    // fd 和 bk 不存在 (用户数据覆盖此区域)
    // user data starts here
};

// 空闲的 chunk
struct malloc_chunk {
    size_t prev_size;
    size_t size;
    struct malloc_chunk *fd; // 指向前一个空闲chunk (bin链表中)
    struct malloc_chunk *bk; // 指向后一个空闲chunk (bin链表中)
    // 大chunk还有 fd_nextsize, bk_nextsize (用于large bin)
};

// size 字段的低3位标志:
// bit 0: PREV_INUSE - 前一个chunk是否在使用中 (不是空闲)
// bit 1: IS_MMAPPED - 此chunk是通过mmap分配的
// bit 2: NON_MAIN_ARENA - 不属于main arena
```

chunk 布局示意:

相邻的两个chunk:





7.4 free() 如何知道要释放的大小

这是经典面试题。答案: `free(ptr)` 传入的 `ptr` 指向用户数据区的起始位置。**chunk头部就在 ptr 之前**, `size` 字段记录了当前 chunk 的大小。

```
// free() 的内部逻辑 (简化)
void free(void *ptr) {
    if (ptr == NULL) return;

    // ptr 指向用户数据区, chunk头部在 ptr - 16 (64位)
    // 或者 ptr - 8 (32位)
    struct malloc_chunk *chunk = mem2chunk(ptr);
    // mem2chunk: (chunk *)((char *)ptr - 2*SIZE_SZ)

    size_t size = chunk->size; // 从这里知道大小!

    // 检查标志位
    if (size & IS_MMAPPED) {
        // mmap分配的内存, 直接munmap
        munmap_chunk(chunk);
        return;
    }

    // 检查相邻的后一个chunk是否空闲, 如果空闲则合并
    // 检查相邻的前一个chunk是否空闲 (通过PREV_INUSE位), 如果空闲则合并

    // 合并后的chunk加入 unsorted bin (或 fast bin)

    // 如果top chunk相邻, 合并进top chunk
}
```

7.5 内存碎片

内存碎片类型:

1. 内部碎片 (Internal Fragmentation)
 - 分配的内存比实际请求的多 (对齐、chunk头开销)
 - 例: 请求 3 字节, 实际分配 32 字节
 - 所有分配器都必然有内部碎片, 程度不同
2. 外部碎片 (External Fragmentation)
 - 空闲内存总量足够, 但不连续, 无法满足大块分配请求

- 例：有 100MB 空闲，但不连续，请求 50MB 的连续内存失败
- 靠 chunk 合并 (coalescing) 来缓解

[Used|FREE|Used|FREE|Used|FREE]
无法分配大于最大单个FREE块的内存

合并后：

[Used| FREEx2 |Used| FREEx2]
通过检查prev/next chunk来合并

减少碎片的方法:

- 使用内存池 (固定大小分配器)
- 使用 slab/slub 分配器 (内核做法)
- 避免频繁的小块分配和释放
- 使用 `mmap()` 分配大块 (释放后立即归还)

8. Copy On Write (COW)

8.1 COW 机制深度剖析

Copy-On-Write (写时复制) 是一种惰性优化技术: 数据在被修改前，多个使用者共享同一份副本；只有当某个使用者企图写入时，才进行真正的复制。

在 Linux 中，COW 广泛应用于:

- `fork()` 后的父子进程地址空间
- `mmap(MAP_PRIVATE)` 的文件映射

8.2 fork + COW 完整流程

Step 1: 父进程调用 `fork()`

父进程地址空间 (简化):

		物理页框
+-----+		
.text (R+X)	---> [代码页 A]	(可共享)
.data (R+W)	---> [数据页 B]	(可写)
heap (R+W)	---> [堆页 C]	(可写)
stack (R+W)	---> [栈页 D]	(可写)
+-----+		

Step 2: `copy_mm()` - 复制页表但不复制物理页

子进程获得新的页表，但页表项指向相同的物理页:

父进程页表		子进程页表
[A:R+X]	-----> [代码页 A]	<----- [A:R+X]
[B:R]	-----> [数据页 B]	<----- [B:R]
[C:R]	-----> [堆页 C]	<----- [C:R]
[D:R]	-----> [栈页 D]	<----- [D:R]

关键：所有可写页的 PTE 都被标记为 "只读" (R/W=0)
并且设置 COW 标记（软件标记，PTE中没有专门的COW位）
引用计数增加：_refcount = 2（每个物理页）

Step 3：父子进程都读，没问题（直接共享）

Step 4：子进程写数据页 B（写入 data[0]）

1. CPU执行写操作
2. MMU检查页表：
 - PTE存在 (P=1)
 - 但 R/W=0（标记为只读!）
 - 用户态写入被拒绝
3. CPU触发 #PF (Page Fault)
4. do_page_fault() 检查：
 - 是合法的地址（属于进程的 VMA）
 - VMA 标记为可写 (VM_WRITE)
 - 但 PTE 标记为只读 -> 这是 COW 页！
5. COW处理：
 - a. 分配一个新物理页 (alloc_page)
 - b. 将原页内容复制到新页 (memcpy)
 - c. 子进程PTE更新：指向新页，权限改为 R+W
 - d. 父进程PTE更新：权限改为 R+W（不再COW）
 - e. 原物理页引用计数：2 -> 1
 - f. 刷新对应TLB项
6. 返回用户态，重新执行写入，成功！

结果：

父进程页表		子进程页表
[A:R+X]	----->	[代码页 A] <----- [A:R+X]
[B:R+W]	----->	[数据页 B] [新数据页 B'] <--- [B':R+W]
[C:R]	----->	[堆页 C] <----- [C:R]
[D:R]	----->	[栈页 D] <----- [D:R]
		(refcount=1) (refcount=1)

8.3 页表项操作的细节

// 内核中 COW 页的处理（简化版）

```
// do_wp_page() - write Protect fault handler (mm/memory.c)
static vm_fault_t do_wp_page(struct vm_fault *vmf) {
    struct vm_area_struct *vma = vmf->vma;
    struct page *old_page = vmf->page;

    // 1. 检查是否确实需要 COW
    if (page_count(old_page) == 1) {
        // 只有一个引用，不需要复制，直接恢复可写
        pte_t entry = pte_mkwrite(pte_mkdirty(vmf->orig_pte));
        set_pte_at(vma->vm_mm, vmf->address, vmf->pte, entry);
        return 0; // 无需COW
    }
}
```

```
// 2. 分配新页
struct page *new_page = alloc_page_vma(GFP_HIGHUSER_MOVABLE, vma, vmf->address);

// 3. 复制原页内容到新页
copy_user_highpage(new_page, old_page, vmf->address, vma);

// 4. 减少原页引用计数 (不再是 COW)
put_page(old_page); // page->_refcount--

// 5. 更新 PTE: 指向新页, 可写, 脏
pte_t entry = pte_mkdirty(pte_mkwrite(pte_mkdirty(mk_pte(new_page, vma->vm_page_prot))));
set_pte_at(vma->vm_mm, vmf->address, vmf->pte, entry);

return 0;
}
```

面试题: vfork 和 fork + COW 的区别?

维度	fork + COW	vmfork
地址空间	有独立地址空间 (通过COW延迟复制)	共享父进程地址空间
执行顺序	不确定 (调度器决定)	保证子进程先执行
父进程状态	同时运行	阻塞, 直到子进程exit或exec
安全性	安全, 不会意外修改父进程数据	危险, 子进程可能破坏父进程数据
现代使用	广泛使用	基本弃用

9. 文件系统

9.1 VFS (Virtual File System) 架构

VFS 是 Linux 内核的一个抽象层, 为上层应用提供统一的文件操作接口, 隐藏下层各种文件系统 (ext4, XFS, NFS, procfs, sysfs 等) 的差异。




```

int (*write_inode)(struct inode *, struct writeback_control *wbc);
int (*sync_fs)(struct super_block *sb, int wait);
};

struct inode_operations {
    int (*create)(struct mnt_idmap *, struct inode *, struct dentry *, umode_t, bool);
    struct dentry *(*lookup)(struct inode *, struct dentry *, unsigned int);
    int (*link)(struct dentry *, struct inode *, struct dentry *);
    int (*unlink)(struct inode *, struct dentry *);
    int (*mkdir)(struct mnt_idmap *, struct inode *, struct dentry *, umode_t);
    int (*rmdir)(struct inode *, struct dentry *);
};

struct file_operations {
    ssize_t (*read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char __user *, size_t, loff_t *);
    int (*mmap)(struct file *, struct vm_area_struct *);
    int (*open)(struct inode *, struct file *);
    int (*flush)(struct file *, fl_owner_t id);
    int (*fsync)(struct file *, loff_t, loff_t, int datasync);
};

```

9.2 inode 深度解析

inode (index node) 是文件系统的核心概念。它存储了关于文件的**元数据** (metadata)，但不包括文件名和文件内容本身。文件名存储在目录项 (dentry) 中，内容存储在数据块中。

```

// ext4 inode 包含的主要信息 (简化)
struct ext4_inode {
    __le16 i_mode;           // 文件类型和权限 (如 S_IFREG | 0644)
    __le16 i_uid;           // 属主UID
    __le32 i_size;          // 文件大小 (低32位)
    __le32 i_atime;         // 最后访问时间 (Access)
    __le32 i_ctime;         // inode修改时间 (Change)
    __le32 i_mtime;         // 内容修改时间 (Modify)
    __le32 i_dtime;         // 删除时间
    __le16 i_gid;           // 属组GID
    __le16 i_links_count;   // 硬链接数
    __le32 i_blocks;        // 文件占用的块数 (512字节为单位)
    __le32 i_flags;         // 标志
    __le32 i_block[15];     // 数据块指针 (见下方详解)
    __le32 i_generation;    // 文件版本 (NFS用)
    __le32 i_size_high;     // 文件大小 (高32位, ext4大文件支持)
};

// 通过 stat 系统调用获取:
// $ stat file.txt
//  File: file.txt
//   Size: 1234      Blocks: 8          IO Block: 4096   regular file
// Device: 802h/2050d Inode: 123456     Links: 1
// Access: (0644/-rw-r--r--)  Uid: (1000/  user)   Gid: (1000/  group)
// Access: 2024-01-15 10:30:00.000000000

```

```
// Modify: 2024-01-15 10:25:00.000000000
// Change: 2024-01-15 10:25:00.000000000
```

注意: 文件名不在 inode 中! 文件名存储在**目录**中。目录本质上是一个特殊的文件，其内容是一系列 (dentry_name, inode_number) 映射。

目录的存储结构:

目录 /home/user/

文件名	inode号
.	1001
..	999
file.txt	2001
photo.jpg	2002

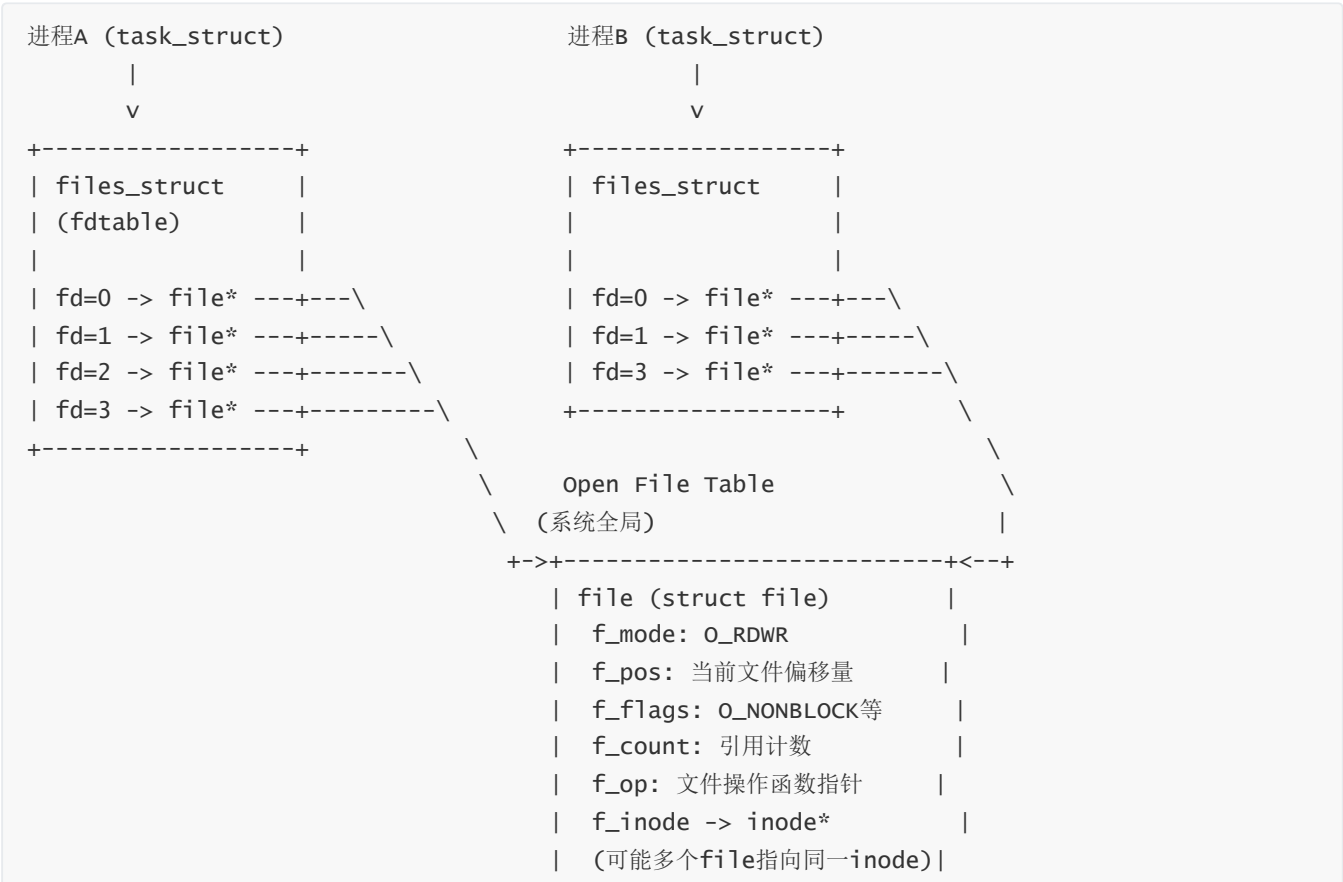
所以删除文件的本质是:

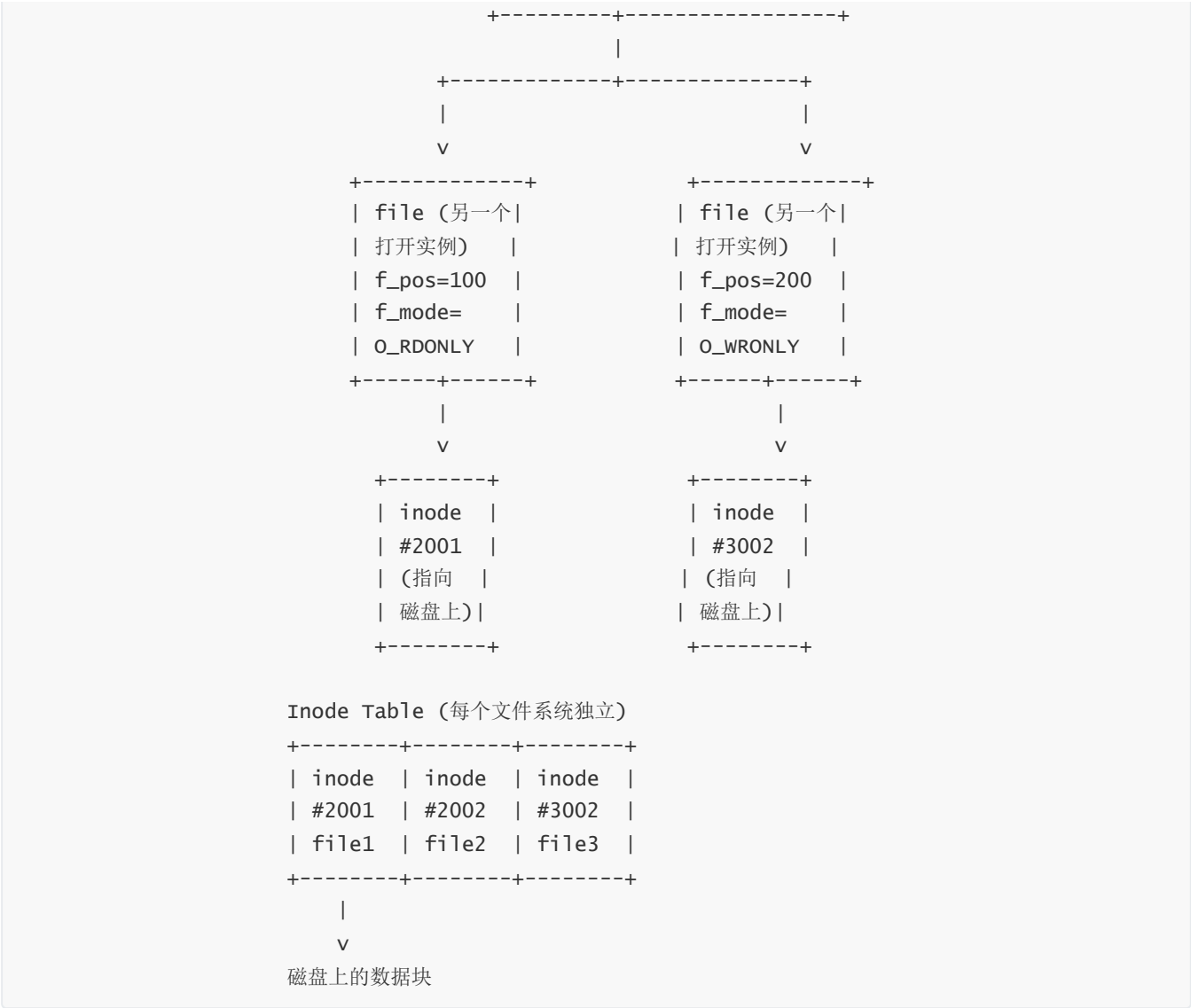
- 1. 从目录中删除 (文件名, inode号) 条目
- 2. inode的 i_links_count 减1
- 3. 如果 i_links_count = 0 且无进程打开, 释放数据块

9.3 文件描述符 (File Descriptor)

文件描述符 fd 是一个整数, 是进程文件描述符表中的索引。但底层的数据结构远比这个整数丰富。

三表结构关系图





关键理解:

层级	数据结构	生命周期	关键字段	说明
进程级	files_struct (fdtable)	随进程	fd[] 数组	每个进程有自己的fd编号空间
系统级	struct file	从open到close	f_pos, f_mode, f_count, f_inode	一个open创建一个file, 但fd可以dup
文件系统级	struct inode	文件存在期间	i_size, i_mode, i_blocks	多个file可能指向同一个inode

dup/dup2 和 fork 对文件描述符的影响:

```
// dup: 复制fd, 新fd指向同一个 struct file
int fd2 = dup(fd); // fd和fd2共享同一个 struct file (f_pos也共享!)

// dup2: 复制并指定新fd号
int fd3 = dup2(fd, 10); // fd和10指向同一个 struct file

// fork: 子进程复制fdtable, 但指向的是同一个 struct file
// 这就是为什么 fork 后父子进程写同一个文件时 f_pos 会相互影响!
```

9.4 ext4 文件系统概览

ext4 磁盘布局 (一个块组):

```
+-----+-----+-----+-----+-----+-----+
| Super | GDT   | Block | Inode | Inode | Data   |
| Block | (组描述符| Bitmap | Bitmap | Table | Blocks |
|       | 表)    |       |       |       |       |
+-----+-----+-----+-----+-----+-----+

```

Super Block: 文件系统元信息 (block大小, inode总数, magic)
 GDT: 块组描述符表
 Block Bitmap: 块使用位图 (每个bit对应一个data block)
 Inode Bitmap: inode使用位图
 Inode Table: 所有inode (每个inode 256字节在ext4中)
 Data Blocks: 实际文件数据

ext4 inode 数据块寻址 (多级索引):

i_block[] 数组 (60字节, 15个32位条目, 对4KB块):

- [0:11] - 直接块指针 (Direct Blocks): 12 个
每块4KB, 共 48KB
- [12] - 间接块指针 (Single Indirect): 1 个
指向一个 4KB块 (包含 1024 个 4字节指针)
寻址: $1024 \times 4KB = 4MB$
- [13] - 双间接块指针 (Double Indirect): 1 个
 $1024 \times 1024 \times 4KB = 4GB$
- [14] - 三间接块指针 (Triple Indirect): 1 个
 $1024 \times 1024 \times 1024 \times 4KB = 4TB$

总寻址能力: $48KB + 4MB + 4GB + 4TB \approx 4TB$
 (ext4还支持 Extents 以提高大文件效率)

Extent Tree (ext4的改进):

直接映射一段连续的块范围:

[logical_block, length, physical_block]

例如: [0, 256, 10000] - 从块0到块255, 映射到物理块10000-10255

比间接块效率高得多 (一个extent条目代替256个间接块条目)

10. IO流程

10.1 完整的读/写流程

read() 完整流程图

```
用户调用 read(fd, buf, 4096) [用户态]
    |
    v
glibc -> syscall [切换内核态]
    |
    v
sys_read() -> ksys_read()
    |
    v
fdget(fd) - 根据fd获取 struct file
    |
    v
file->f_op->read() - 调用文件系统相关的read
    对于普通文件: generic_file_read_iter()
        |
        v
+----> 检查 Page Cache
    |   find_get_page() / filemap_get_pages()
    |   页是否已经在内存中?
    |   |
    |   Yes | No
    |   |   |
    |   |   |
    |   |   v
    |   |   page_cache_sync_readahead() - 预读
    |   |   (Read-Ahead, 读比请求更多的数据到cache)
    |   |   |
    |   |   v
    |   |   filemap_get_pages() - 再次尝试
    |   |   |
    |   |   v
    |   |   页不在 -> 触发缺页
    |   |   filemap_fault():
    |   |   |
    |   |   v
    |   |   分配 page (alloc_page)
    |   |   加入 page cache (add_to_page_cache_lru)
    |   |   |
    |   |   v
    |   |   提交 BIO (Block I/O) 请求
    |   |   submit_bio():
    |   |   上层: 文件系统生成 bio
    |   |   中层: Block Layer (bio -> request)
    |   |   下层: I/O调度器 (合并, 排序)
    |   |   设备驱动: 发命令给磁盘
    |   |   磁盘: DMA到内存
```

```

|           | 中断：BIO 完成
|           | |
|           | v
|           | 页现在在内存中了
|           +<---+
|           |
|           v
copy_to_user(buf, page, 4096) - 从内核Page复制到用户buf
|
| v
更新 file->f_pos += 已读取字节数
|
| v
fdput(fd)
|
| v
sysret [切换回用户态]
|
| v
返回已读取字节数

```

write() 完整流程图

```

用户调用 write(fd, buf, 4096) [用户态]
|
| v
glibc -> syscall [切换内核态]
|
| v
sys_write() -> ksys_write()
|
| v
fdget(fd) - 根据fd获取 struct file
|
| v
file->f_op->write() - 文件系统相关write
对于普通文件: generic_file_write_iter()
|
| v
copy_from_user(kernel_buf, buf, 4096) - 用户数据 -> 内核
|
| v
查找/分配 Page Cache 中的页
|
| v
将数据写入 Page Cache (标记为 dirty)
set_page_dirty()
|
| v
更新 file->f_pos += 已写入字节数
|
| v

```

```

fdput(fd)
    |
    v
sysret [回到用户态]
    |
    v
write() 返回成功!

```

? 数据什么时候真正到磁盘?

- 不是write()时! write()只是写到 Page Cache

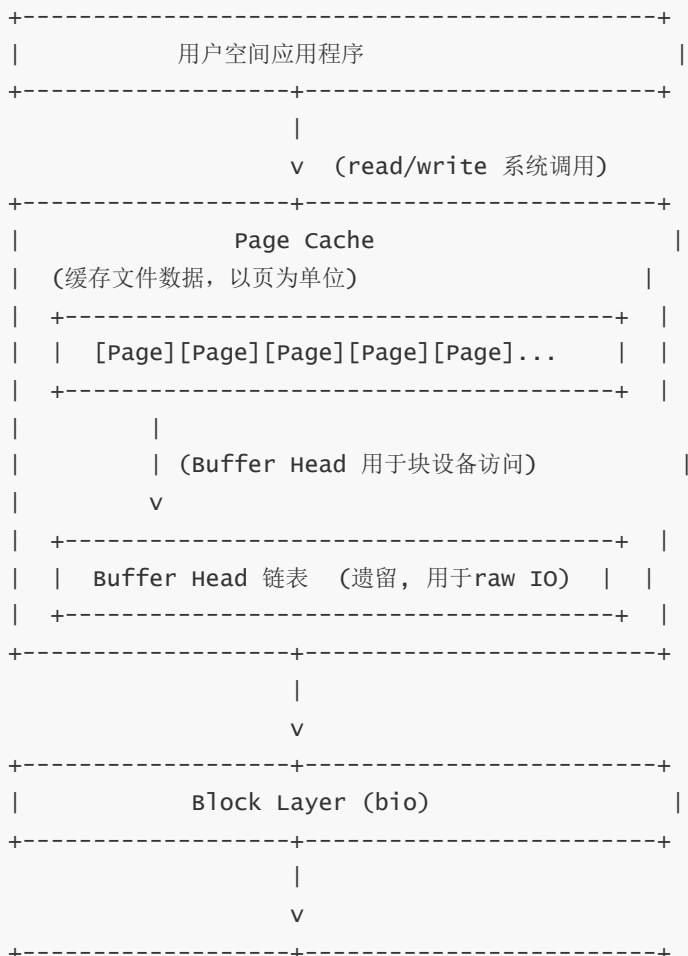
数据落盘的时机:

1. 用户调用 fsync() / fdatasync() / sync() 强制刷盘
2. 内核pdflush/flush线程周期性刷脏页
(/proc/sys/vm/dirty_writeback_centisecs, 默认 500 = 5s)
3. 脏页比例超过阈值时强制写回
(/proc/sys/vm/dirty_ratio, 默认 20% 的总内存)
4. 内存紧张时 (内存回收 - kswapd)

10.2 Buffer Cache 与 Page Cache

在现代 Linux (2.4+) 中, Buffer Cache 和 Page Cache 已经**统一**。Page Cache 以页 (4KB) 为单位缓存文件内容, Buffer Cache 现在实际上是 Page Cache 的一部分。

缓存层次结构:



| 磁盘 / SSD |

+-----+

Page Cache 的回收:

- LRU (Least Recently Used) 双链表
 - + Active List: 最近访问过的页
 - + Inactive List: 很久没访问的页
- kswapd 线程负责回收
- /proc/sys/vm/swappiness 控制

10.3 Direct IO vs Buffered IO

维度	Buffered IO (标准IO)	Direct IO
缓存	使用 Page Cache	绕过 Page Cache
对齐	不需要	用户buffer必须按512/4096对齐
数据一致性	write后数据在cache, 不一定在磁盘	write后数据直接在磁盘
性能	读快 (缓存命中), 写快 (异步写回)	读写每次都走磁盘
适用场景	通用文件读写	数据库 (自己有缓存机制), 流媒体
O_DIRECT	不设置	设置 O_DIRECT 标志
与O_SYNC结合	O_SYNC: write后等待写入磁盘	O_DIRECT+O_SYNC: 最严格的同步写

```
// 普通IO
int fd = open("file.txt", O_RDWR);
write(fd, buf, size); // 数据进了 Page Cache

// Direct IO
int fd = open("file.txt", O_RDWR | O_DIRECT);
// buf 必须是对齐的!
void *buf;
posix_memalign(&buf, 512, size); // 或 4096 对齐
write(fd, buf, size); // 直接发 BIO 到磁盘
```

面试重点

高频考点

1. **fork() 和 Copy-On-Write** — 几乎必问。必须能画出COW的流程, 解释为什么fork后写页会触发缺页异常。
2. **虚拟内存和分页** — 虚拟地址到物理地址的翻译过程, 多级页表, TLB的作用。
3. **进程和线程的区别** — 不仅是概念层面, 还要知道Linux内核中的统一实现 (clone 标志)。
4. **malloc/free 内部实现** — free 怎么知道要释放多少内存 (通过 chunk header 的 size 字段)。
5. **系统调用过程** — 从用户态到内核态再到用户态的完整路径。
6. **文件描述符的三表结构** — fd -> file -> inode 的对应关系, dup/fork 的影响。

7. **僵尸进程和孤儿进程** — 产生原因、危害、解决方案。
8. **Buffer IO vs Direct IO** — 数据何时真正落盘，Page Cache 的作用。

常见陷阱

1. **混淆 fork 返回值的含义** — 父进程返回子进程PID (>0)，子进程返回0，-1 表示失败。
2. **以为 write() 后数据一定在磁盘上** — write() 只写到 Page Cache，fsync() 才刷盘。
3. **不知道 free(ptr) 怎么知道大小** — chunk header 在 ptr 前面存储了 size。
4. **混淆进程和线程的上下文切换开销** — 线程切换不需要切换页表 (不写CR3)，所以更快。
5. **忘记 COW 是惰性优化** — fork 后不是立刻复制，是写的时候才复制。
6. **误解 inode 存储文件名** — inode 不存文件名，文件名存在目录的 dentry 中。
7. **混淆 buf 和 cache** — Linux 中 buffer 和 cache 在 /proc/meminfo 中的含义与传统不同 (buffer: 块设备的元数据，cache: 文件的页缓存)。
8. **不知道 CFS 使用红黑树** — CFS 不是简单的轮转，是用红黑树按 vruntime 排序。

建议背诵内容

必须能脱稿画出的图:

- 进程状态转换图 (R/S/D/T/Z 的状态转移)
- 虚拟地址翻译流程 (CR3 -> PML4 -> PDPT -> PD -> PT -> Page)
- 文件描述符三表结构 (fdtable -> file -> inode)
- fork + COW 流程图
- 系统调用端到端流程图
- Page Fault 处理流程图

必须能脱稿回答的问题:

1. 进程和线程的区别是什么? Linux 内核如何实现线程?
 2. fork() 具体做了哪些事情? COW 如何工作?
 3. 虚拟内存的作用是什么? 多级页表为什么必要?
 4. malloc(100) 内部发生了什么? free(ptr) 怎么知道释放多少?
 5. 一个 read() 系统调用从开始到结束经历了什么?
 6. 什么是僵尸进程? 如何避免?
 7. CFS 调度器的核心思想是什么? 为什么用红黑树?
-

补充专题

A. 信号 (Signal) 机制

信号是 Linux 中**异步进程间通信**的核心机制。当某个事件发生时，内核向目标进程发送一个信号，目标进程可以注册处理函数或忽略该信号。

信号生命周期

信号的生命周期：

1. 产生 (Generation)
- 硬件异常：SIGSEGV, SIGFPE
 - 终端按键：SIGINT (Ctrl+C), SIGTSTP (Ctrl+Z)
 - 系统调用：kill(), raise(), alarm()
 - 软件事件：SIGCHLD (子进程退出), SIGPIPE (管道破裂), SIGALRM (定时器到期)
2. 递送 (Delivery)
- 内核在目标进程从内核态返回用户态时检查 pending 信号
 - 在 `do\_signal()` 中处理
 - 时机：系统调用返回时 / 中断返回时 / 调度后
3. 处理 (Handling)
- SIG_DFL (默认)：终止/忽略/停止/继续 (取决于信号类型)
 - SIG_IGN (忽略)：丢弃信号
 - 自定义处理函数：sigaction 注册的函数
 - SIGKILL 和 SIGSTOP：不能捕获/忽略/阻塞！

信号在进程中的存储：

```
task_struct->pending (私有信号)
task_struct->signal->shared_pending (线程组共享信号)
task_struct->blocked (阻塞信号集, sigset_t)
```

信号递送的时机：

```
ret_to_user() / prepare_exit_to_usermode()
-> exit_to_usermode_loop()
-> do_signal() // 检查并处理 pending 信号
```

常见信号速查表

信号	编号	默认行为	说明	是否可捕获
SIGHUP	1	Term	终端断开/控制进程退出	是
SIGINT	2	Term	中断 (Ctrl+C)	是
SIGQUIT	3	Core	退出 (Ctrl+) + core dump	是
SIGILL	4	Core	非法指令	是
SIGABRT	6	Core	abort() 调用	是

信号	编号	默认行为	说明	是否可捕获
SIGFPE	8	Core	浮点异常 (除0)	是
SIGKILL	9	Term	强制杀死 (不可捕获)	否
SIGSEGV	11	Core	段错误 (非法内存访问)	是
SIGPIPE	13	Term	向已关闭的管道写入	是
SIGALRM	14	Term	alarm() 定时器到期	是
SIGTERM	15	Term	终止 (kill 默认)	是
SIGSTOP	19	Stop	暂停 (不可捕获)	否
SIGTSTP	20	Stop	终端暂停 (Ctrl+Z)	是
SIGCHLD	17	Ign	子进程状态变化	是
SIGUSR1	10	Term	用户自定义1	是
SIGUSR2	12	Term	用户自定义2	是

信号处理代码示例

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

// 信号处理函数 (必须是 async-signal-safe!)
volatile sig_atomic_t g_flag = 0;

void sigint_handler(int signo) {
    // 注意: 处理函数中只能调用 async-signal-safe 的函数
    // printf 不是 async-signal-safe! 这里仅作为演示
    g_flag = 1; // sig_atomic_t 的写入是原子的
}

int main() {
    struct sigaction sa;
    sa.sa_handler = sigint_handler;
    sa.sa_flags = SA_RESTART; // 被信号中断的系统调用自动重启
    sigemptyset(&sa.sa_mask);
    sigaction(SIGINT, &sa, NULL);

    // 屏蔽 SIGINT
    sigset_t block_mask, old_mask;
    sigemptyset(&block_mask);
    sigaddset(&block_mask, SIGINT);
    sigprocmask(SIG_BLOCK, &block_mask, &old_mask);
    // ... 临界区 (SIGINT不会被递送) ...
    sigprocmask(SIG_SETMASK, &old_mask, NULL);
}
```

```
// 同步等待信号 (不用忙等)
sigset_t wait_mask;
sigemptyset(&wait_mask);
sigaddset(&wait_mask, SIGINT);
int sig;
sigwait(&wait_mask, &sig); // 阻塞直到收到SIGINT
printf("Got signal %d\n", sig);

return 0;
}
```

面试题: 什么是可重入函数 (Reentrant) 和 异步信号安全 (Async-Signal-Safe)?

答:

- **可重入函数:** 函数可以同时多个上下文中安全执行 (如被多个线程调用或中断处理中调用), 不使用全局/静态变量, 或使用锁保护。 `strtok()` 不可重入 (用静态变量), `strtok_r()` 可重入。
- **异步信号安全函数:** 可以在信号处理函数中安全调用的函数。因为信号处理函数可能在程序的任意点被调用 (可能正处于 `malloc` 的中间状态!), 所以只能使用保证不会死锁/破坏状态的函数。典型的 `async-signal-safe` 函数: `write()`, `read()`, `_exit()`, `signal()`, `sigprocmask()` 等。 `printf()`, `malloc()` 不是 `async-signal-safe`!

B. 进程间通信 (IPC) 全景

IPC 方式	数据传输	持久性	同步机制	使用场景
管道 (Pipe)	字节流	随进程	读写阻塞	父子进程通信 (<code>ls grep txt</code>)
命名管道 (FIFO)	字节流	随文件	读写阻塞	无关进程通信
信号 (Signal)	少量信息	无	异步	通知事件 (SIGCHLD, SIGTERM)
消息队列 (POSIX MQ)	消息 (带优先级)	随内核	阻塞/非阻塞	结构化消息传递
共享内存 (SHM)	最快 (直接内存)	随内核	无 (需额外同步)	大数据量、低延迟通信
信号量 (Semaphore)	计数器	随内核	原子操作	进程间同步互斥
Socket	字节流/数据报	可跨网络	可选	网络通信, 本地通信 (Unix Domain)
mmap (文件映射)	内存映射	随文件	可选	进程间共享大文件数据

共享内存深入

```
// POSIX 共享内存示例
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
```

```

// 创建共享内存
int fd = shm_open("/my_shm", O_CREAT | O_RDWR, 0666);
ftruncate(fd, 4096); // 设置大小

// 映射到进程地址空间
void *ptr = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);

// 直接读写 ptr
sprintf((char *)ptr, "Hello from Process A!");

// 另一个进程
int fd2 = shm_open("/my_shm", O_RDWR, 0666);
void *ptr2 = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd2, 0);
printf("%s\n", (char *)ptr2); // 读到了!

// 清理
munmap(ptr, 4096);
close(fd);
shm_unlink("/my_shm");

```

注意: 共享内存是最快的 IPC (数据不需要在内核和用户态之间复制), 但需要额外的同步机制 (如 POSIX 信号量、互斥锁在共享内存中) 来避免竞态条件。

C. Epoll 核心数据结构速览

(详细内容见第五章, 这里只是内核数据结构的概述)

epoll 的内核数据结构:

1. eventpoll (每个 epoll fd 对应一个)

```

struct eventpoll {
    struct rb_root rbr;    // 红黑树根 (存储所有监听的 fd)
    struct list_head rdllist; // 就绪链表 (就绪的 fd)
    wait_queue_head_t wq; // epoll_wait 的等待队列
    ...
};

```

2. epitem (每个被监听的 fd 对应一个)

```

struct epitem {
    struct rb_node rbn;    // 红黑树节点
    struct list_head rdllink; // 就绪链表节点
    struct epoll_filefd ffd; // {fd, file*}
    struct eventpoll *ep;   // 所属的 eventpoll
    struct epoll_event event; // 用户注册的事件
    ...
};

```

epoll_ctl(EPOLL_CTL_ADD):

- 创建 epitem, 插入 eventpoll 的红黑树

- 在目标文件的等待队列上注册回调 (`ep_poll_callback`)

`epoll_wait()`:

- 检查 `rdllist` 是否有就绪项
- 如果没有: 将当前进程加入 `eventpoll.wq`, 进入睡眠
- 有: 将就绪事件拷贝到用户空间

数据就绪时 (网卡收到数据 -> 协议栈处理 -> `socket` 缓冲区有数据):

调用 `ep_poll_callback()`
-> 将对应的 `epitem` 加入 `rdllist`
-> 唤醒 `eventpoll.wq` 上等待的进程
-> `epoll_wait` 返回

为什么 `epoll` 快?

- `O(1)` 获取就绪事件 (只需读取 `rdllist`)
- 事件驱动回调, 不需要轮询
- `mmap` 减少了内核到用户态的数据拷贝 (早期实现)

D. 内核模块与 eBPF (概述)

内核模块

```
// 最简单的内核模块 (hello.c)
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Your Name");

static int __init hello_init(void) {
    printk(KERN_INFO "Hello, kernel!\n");
    return 0;
}

static void __exit hello_exit(void) {
    printk(KERN_INFO "Goodbye, kernel!\n");
}

module_init(hello_init);
module_exit(hello_exit);

// 编译: Makefile
// obj-m += hello.o
// make -C /lib/modules/$(uname -r)/build M=$(PWD) modules

// 操作:
// sudo insmod hello.ko    (加载)
// sudo rmmod hello       (卸载)
// dmesg | tail            (查看输出)
```

eBPF (extended Berkeley Packet Filter)

eBPF 是 Linux 内核中的**安全沙箱虚拟机**，允许用户编写程序在内核中安全运行，不修改内核源码。应用场景: 网络优化 (Cilium)、性能分析 (BCC/bpftrace)、安全 (seccomp, Falco)、可观测性 (Pixie)。

eBPF 程序流程：

1. 用户编写 eBPF 程序 (C 语言子集)
2. 编译为 eBPF 字节码 (`clang -target bpf`)
3. 通过 `bpf()` 系统调用加载到内核
4. 内核验证器 (verifier) 检查：
 - 没有无限循环
 - 没有非法内存访问
 - 指令数量有限制
 - 不能调用任意内核函数
5. 验证通过后，eBPF 程序被 JIT 编译为本地机器码
6. 附加到内核钩子点 (kprobe, tracepoint, socket, XDP 等)
7. 触发钩子时执行 eBPF 程序
8. 通过 BPF map (哈希表/数组等) 与用户空间交换数据

E. CFS 调度器补充细节

CFS 的时间记账

```
// CFS 使用 vruntime 而非实际物理时间进行调度

// 调度实体的 vruntime 更新：
vruntime += delta_exec * (NICE_0_LOAD / se->load.weight);

// NICE_0_LOAD = 1024 (nice=0 对应权重)
// nice = -20 时 weight ≈ 88761 -> delta_exec * 0.0115 (vruntime 增长极慢)
// nice = +19 时 weight ≈ 15 -> delta_exec * 68.27 (vruntime 增长极快)
// nice = 0 时 weight = 1024 -> delta_exec * 1.0

// nice值 -20 的进程 相比 nice值 +19 的进程：
// CPU时间比例 = 88761 / 15 ≈ 5917 倍！
// (实际受 sysctl sched_latency 和 sched_min_granularity 调节)
```

调度域和负载均衡

多核系统中的负载均衡（SMP）：

调度域层级（4-socket NUMA）：

Level 0: SMT (HyperThreading, 同一物理核的兄弟)

Level 1: MC (Multi-Core, 同一CPU die 上的核心)

Level 2: DIE (同一 socket/package)

Level 3: NUMA (跨 socket, 通过互联)

load_balance() 在每个 tick 中检查是否需要均衡：

- 某个 CPU 的 runqueue 过载，另一个 CPU 空闲
- 从最忙的 CPU 迁移任务到最闲的 CPU
- 迁移开销：同级域 < 跨域（为了缓存局部性）

F. 内存管理补充

NUMA 架构与内存分配

NUMA (Non-Uniform Memory Access)：

Socket 0

Socket 1

+-----+				+-----+			
CPU0	CPU1	CPU2	CPU3	CPU4	CPU5	CPU6	CPU7
QPI/							
Memory Controller <----->				Memory Controller			
UPI							
DIMM DIMM				DIMM DIMM			
+-----+				+-----+			

- 本地内存访问：快
- 远端内存访问：慢（通过互联总线）
- numa_node 记录每个页属于哪个 NUMA 节点

内核中的 NUMA 感知：

- GFP_KERNEL：优先分配当前节点的内存
- __GFP_THISNODE：强制在当前节点分配
- numactl 工具绑定进程到特定 NUMA 节点
- /proc/sys/vm/zone_reclaim_mode：控制 NUMA 回收策略

伙伴系统 (Buddy System) 和 Slab 分配器

Linux 物理页分配的两个层级：

1. 伙伴系统 (Buddy System) – 分配物理页（以页为单位，2^n 页）

- 目的：减少外部碎片
- 11 个 free_area (arm64: order 0-10)
- order 0: 1 page (4KB)
- order 1: 2 pages (8KB)
- order 2: 4 pages (16KB)
- ...

order 10: 1024 pages (4MB)

- 分配: 找最小能满足的 `order`, 找不到则向上级拆分
- 释放: 检查 `buddy` 是否空闲, 如果是则合并成更大块
- `/proc/buddyinfo` 查看各 `order` 的空闲块数

2. slab/slub/slob 分配器 - 分配内核对象 (以字节为单位)

- `kmalloc()` / `kfree()` 底层使用
- 缓存常用大小的对象
- 每个 `slab` 包含多个对象, 预分配减少碎片
- `/proc/slabinfo` 查看 `slab` 缓存
- `cat /proc/slabinfo | head -20`

常见 `slab` 缓存:

`kmalloc-8`, `kmalloc-16`, `kmalloc-32`, ..., `kmalloc-8192`
`task_struct_cache`, `mm_struct_cache`, `inode_cache`, `dentry_cache`

/proc/meminfo 解读

```
$ cat /proc/meminfo
```

```
MemTotal:      16384000 kB # 总物理内存
MemFree:       2048000 kB # 完全未使用的物理内存
MemAvailable:  10240000 kB # 可分配给新进程的内存 (含可回收的)
Buffers:       512000 kB # 块设备的元数据缓存 (Buffer Cache)
Cached:        8192000 kB # 文件数据的页缓存 (Page Cache)
SwapCached:    0 kB # 被换出又在内存中的页
Active:        6144000 kB # 活跃页 (最近被访问)
Inactive:      4096000 kB # 不活跃页 (可被回收)
SwapTotal:     8192000 kB # 交换空间总大小
SwapFree:      8192000 kB # 空闲交换空间
```

关键计算公式:

实际可用内存 $\approx$ `MemFree` + `Cached` + `Buffers` - 不可回收部分
(`MemAvailable` 已经帮你算好了这个值)

已使用内存 = `MemTotal` - `MemFree` - `Cached` - `Buffers`

G. 中断处理补充 (硬中断/软中断/中断下半部)

中断处理的两半 (`Top Half` / `Bottom Half`):

`Top Half` (硬中断处理程序):

- 响应硬件中断, 快速执行
- 关中断运行 (或高优先级)
- 只做必要的事: 读寄存器、应答设备
- 触发 `Bottom Half`, 然后返回

`Bottom Half` (下半部机制):

1. Softirq (软中断)

- 静态定义, 编译时确定
- `NET_RX_SOFTIRQ`: 网络接收
- `NET_TX_SOFTIRQ`: 网络发送
- `TASKLET_SOFTIRQ`: `tasklet`

- `TIMER_SOFTIRQ`: 定时器
- 同类型软中断可在多CPU上同时运行
- `ksoftirqd` 线程在后台处理

2. Tasklet

- 基于软中断实现
- 同一 `tasklet` 不会同时在多CPU上运行
- 比软中断更简单，更常用

3. workqueue (工作队列)

- 在进程上下文运行 (可睡眠!)
- 适合需要阻塞等待的操作
- `kworker` 线程执行

对比:

`Hard IRQ > Softirq > Tasklet > workqueue` (按严格程度)

不能睡眠 不能睡眠 不能睡眠 可以睡眠

查看中断:

```
$ cat /proc/interrupts      # 硬中断统计
$ cat /proc/softirqs        # 软中断统计
```

H. 系统性能调优补充

进程/线程数限制

```
# 查看限制
$ ulimit -a                # 用户级限制
$ ulimit -u                # 最大进程数
$ ulimit -n                # 最大文件描述符数

# 系统级限制
$ cat /proc/sys/kernel/pid_max          # 最大 PID (默认 32768)
$ cat /proc/sys/kernel/threads-max      # 系统最大线程数
$ cat /proc/sys/fs/file-max             # 系统最大文件描述符数
$ cat /proc/sys/fs/nr_open              # 单进程最大 fd 数

# 修改
$ ulimit -n 65535                # 临时修改
$ echo 1000000 > /proc/sys/kernel/pid_max
```

TCP 内核参数速查

```
# 关键 TCP 参数
net.core.somaxconn = 4096          # listen backlog 的上限
net.ipv4.tcp_max_syn_backlog = 8192 # 半连接队列大小
net.ipv4.tcp_tw_reuse = 1          # TIME_WAIT 端口复用
net.ipv4.tcp_fin_timeout = 30      # TIME_WAIT 时长
net.ipv4.tcp_keepalive_time = 7200  # KeepAlive 空闲时间(秒)
net.ipv4.tcp_keepalive_intvl = 75   # KeepAlive 探测间隔
net.ipv4.tcp_keepalive_probes = 9   # KeepAlive 探测次数
net.core.rmem_max = 16777216        # 最大接收缓冲区
```

```
net.core.wmem_max = 16777216          # 最大发送缓冲区
net.ipv4.tcp_rmem = "4096 87380 16777216" # min/default/max
net.ipv4.tcp_wmem = "4096 65536 16777216"
net.ipv4.tcp_congestion_control = cubic
net.ipv4.tcp_fastopen = 3              # TCP Fast Open
```

文件系统IO调优

```
# IO 调度器 (Linux 5.x+)
$ cat /sys/block/sda/queue/scheduler
[mq-deadline] none

# 调度器类型：
# none          - 多队列 (NVMe 推荐)
# mq-deadline   - 多队列版 deadline (SSD/SATA)
# kyber         - 基于延迟目标 (适合低延迟场景)
# bfq           - 按进程公平分配带宽 (桌面/交互场景)

# 预读 (read-ahead) 调整
$ blockdev --getra /dev/sda
256    (256 扇区 = 128KB)

# 脏页参数
vm.dirty_ratio = 20          # 脏页达总内存20%强制写回
vm.dirty_background_ratio = 10 # 脏页达10%后台写回
vm.dirty_writeback_centisecs = 500 # 写回间隔 (500=5s)
```

下一章: [第四章 TCP/IP协议栈](#) — 网络协议栈、TCP三次握手四次挥手、拥塞控制等。